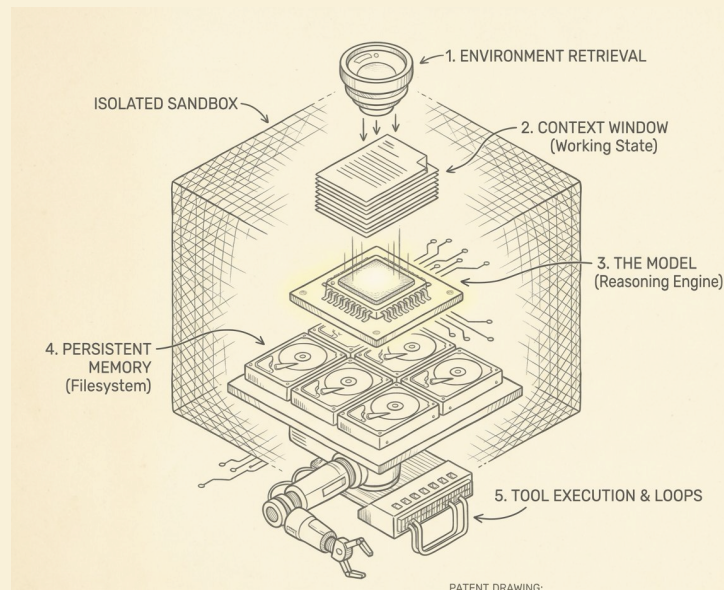


Engineering the Harness: A Practical Workshop

Rajiv Shah
@rajistics
OpenHands

<https://github.com/rajshah4/harness-engineering>



Engineering the Harness:

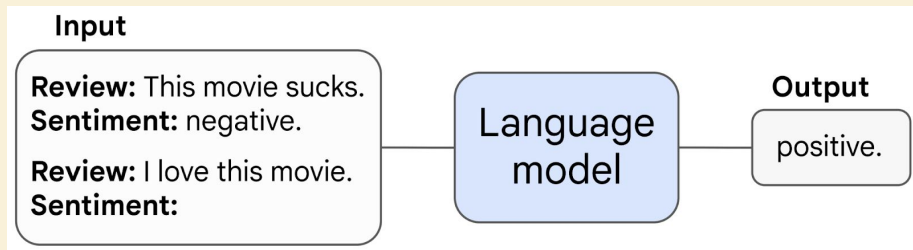
A Practical Workshop

Rajiv Shah
@rajistics
OpenHands

<https://github.com/rajshah4/harness-engineering>



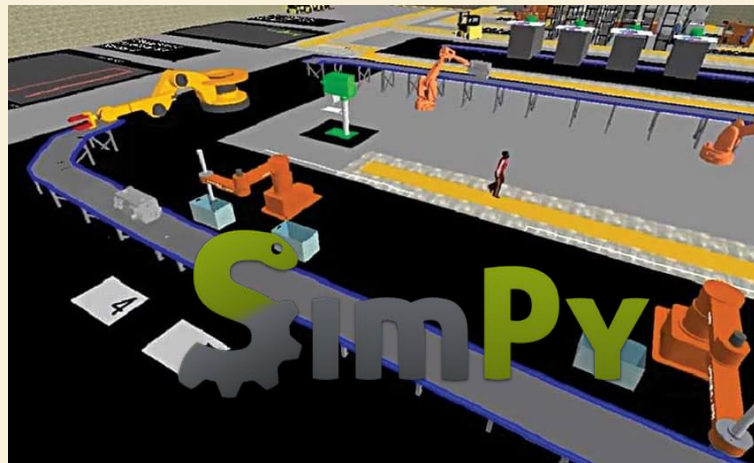
The tasks changed.



2023

Prompting for sentiment

30 tokens - ~0.2 seconds



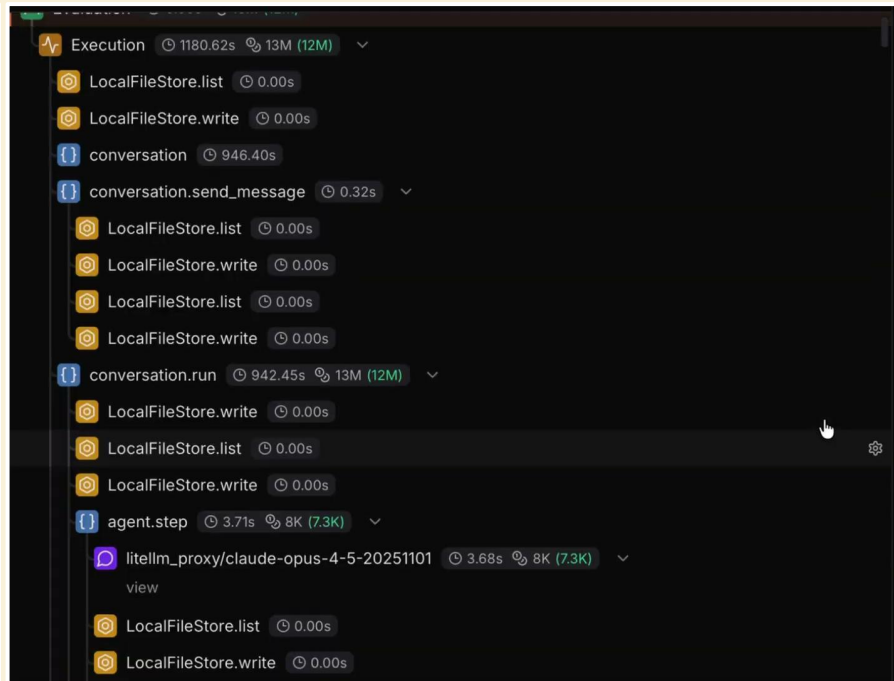
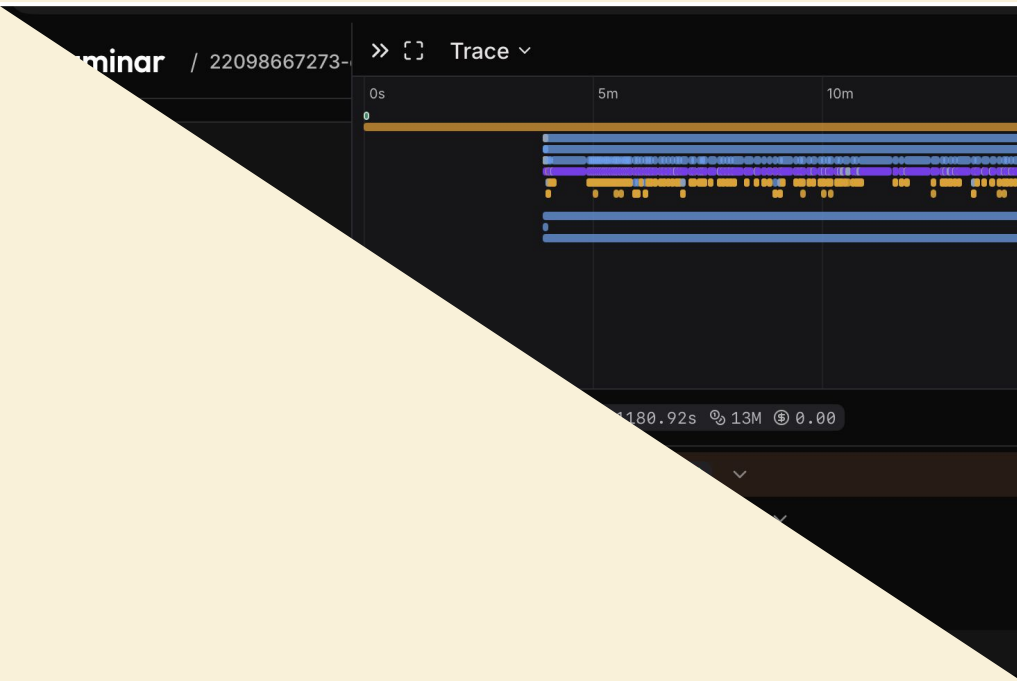
2026

Fix the bug, here is the code
base and test suites

Opus 4.6

12M tokens and 20 minutes

The model isn't solving the problem. The system is.



13M Tokens for 50k Output! and takes 20 minutes

Hi, I'm Rajiv, and this is a masterclass on Harnesses.

Rajiv Shah

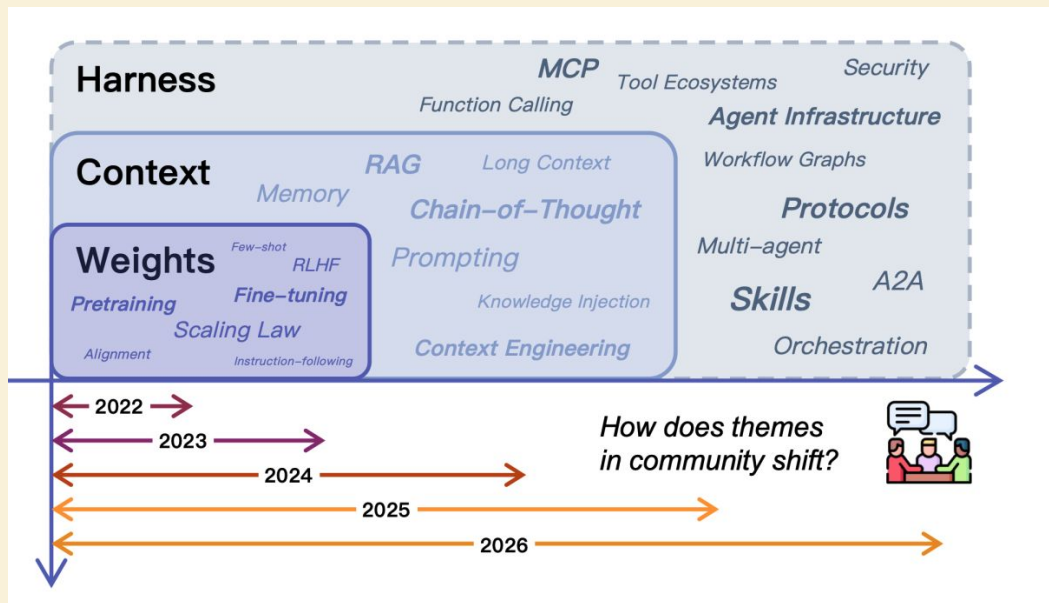
Agentic AI Engineer

OpenHands

Social Media: @rajistics

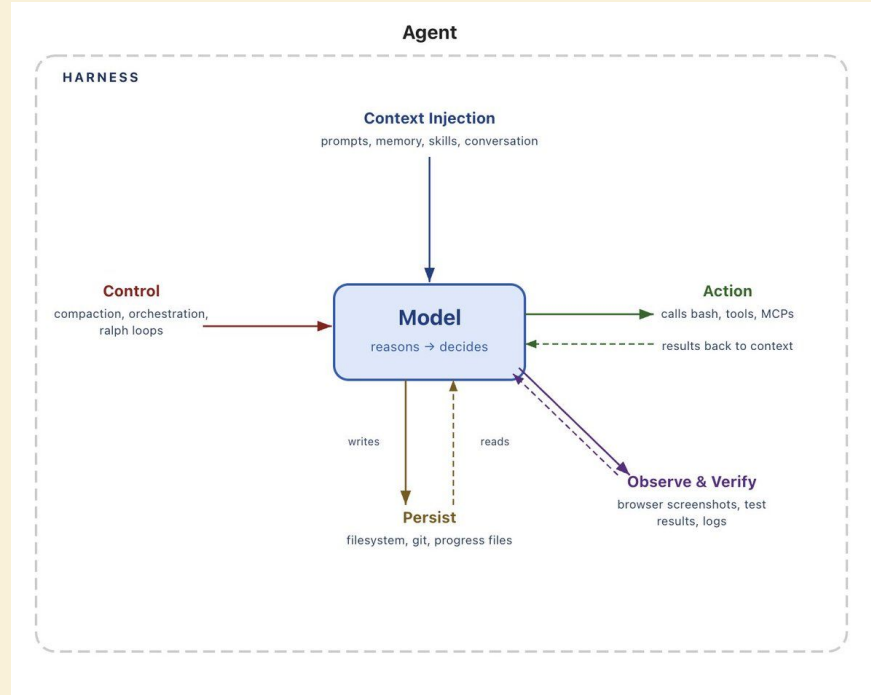


This has been an evolution



LLM Agents

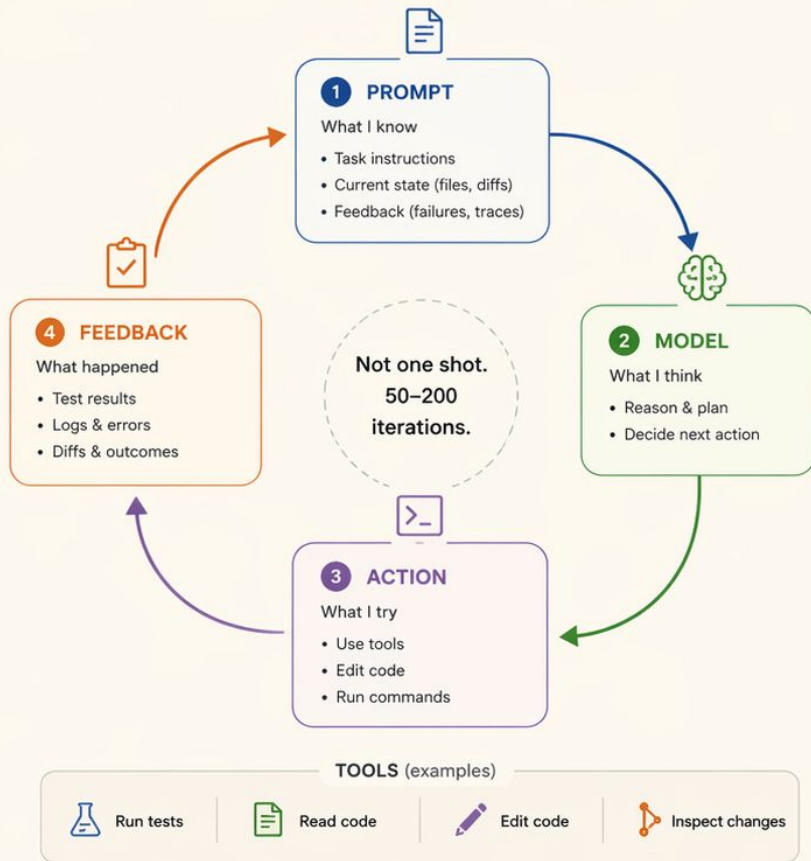
What is in a harness?



<https://blog.langchain.com/the-anatomy-of-an-agent-harness/>

What a modern coding agent actually looks like.

A continuous loop that turns state into progress.



What a modern coding agent actually looks like.

A continuous loop that turns state into progress.



A harness is everything outside the model.

The harness provides the environment, control, and feedback that make the loop reliable and effective.



A good SDK abstracts the harness for agentic actions



```
from openhands.agent import Agent
from openhands.runtime import DockerWorkspace
from openhands.conversation import Conversation

# 1. Create workspace (repo environment)
workspace = DockerWorkspace(
    image="swebench:latest",
    working_dir="/workspace/repo"
)

# 2. Initialize agent (the harness entrypoint)
agent = Agent(
    model="anthropic/claude-3",
    tools=["bash", "editor", "git"],
    system_prompt_kwargs={"cli_mode": True},
)

# 3. Wrap in a conversation loop (the core harness)
conversation = Conversation(
    agent=agent,
    workspace=workspace,
    max_iterations=50,
)
```

```
# 4. Construct the SWE task (benchmark prompt)
task_prompt = f"""
You are given a GitHub issue and a repository.
```

```
Issue:
{issue_text}
```

```
Your goal:
- Identify the bug
- Modify the code (not tests)
- Run tests to verify the fix
- Produce a valid patch
```

```
Work step by step using tools.
"""
```

```
# 5. Run the harness loop
conversation.run(task_prompt)
```

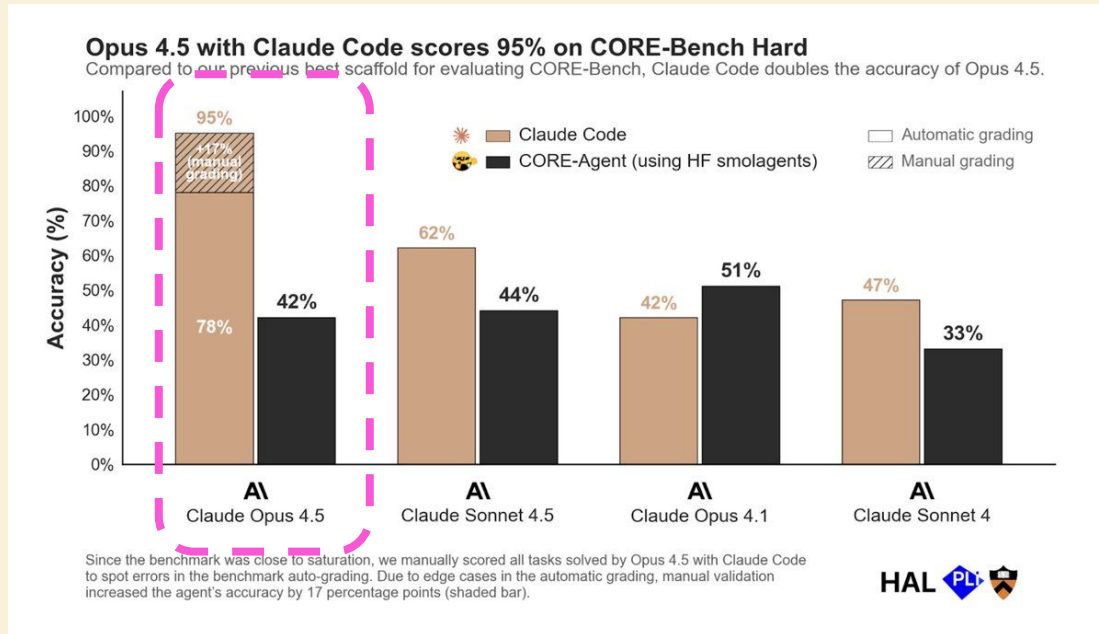
```
# 6. Extract result (the "work product")
patch = workspace.get_git_diff()
print(patch)
```

Same model, 2× performance gap

The change:

95% - Claude Code

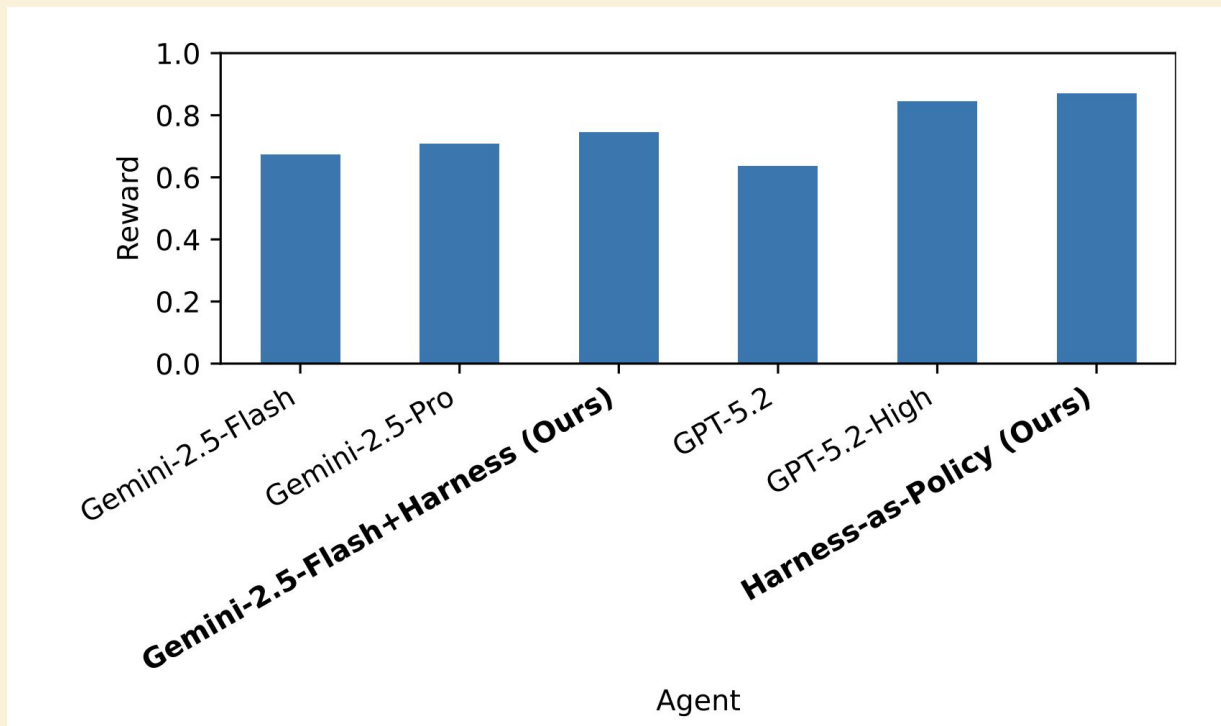
42% - HF smolagents



Everyone on the leaderboard uses the same model

terminal-bench	run	terminal-bench	leaderboard	benchmarks	contributors	news	discord
☐	Rank	Agent	Model	Date	Agent Org	Model Org	Accuracy
☐	4	ForgeCode	Claude Opus 4.6	2026-03-12	ForgeCode	Anthropic	79.8% ± 1.6
☐	8	Capy	Claude Opus 4.6	2026-03-12	Capy	Anthropic	75.3% ± 2.4
☐	11	Terminus-KIRA	Claude Opus 4.6	2026-02-22	KRAFTON AI	Anthropic	74.7% ± 2.6
☐	14	TongAgents	Claude Opus 4.6	2026-02-22	Bigai	Anthropic	71.9% ± 2.7
☐	15	Junie CLI	Multiple	2026-03-07	JetBrains	Multiple	71.0% ± 2.9
☐	17	Droid	Claude Opus 4.6	2026-02-05	Factory	Anthropic	69.9% ± 2.5
☐	20	Crux	Claude Opus 4.6	2026-02-23	Roam	Anthropic	66.9% ± N/A
☐	22	Mux	Claude Opus 4.6	2026-02-13	Coder	Anthropic	66.5% ± 2.5
☐	28	Terminus 2	Claude Opus 4.6	2026-02-06	AfterQuery	Anthropic	62.9% ± 2.7
☐	40	Claude Code	Claude Opus 4.6	2026-02-07	Anthropic	Anthropic	58.0% ± 2.9

Small model + good harness > big model



What harness do you use?

Harness carries a lot of decisions

Claude Code vs Codex CLI: Harness & Experience Comparison		
Dimension	 Claude Code Anthropic	 Codex CLI OpenAI
 Primary Emphasis	Polish, safe defaults, integrated experience	Openness, transparency, flexibility
 Core Philosophy	Curated experience with guardrails and conventions	Open and inspectable; user in control of the stack
 Project Instructions	CLAUDE.md (hierarchical, auto-detected)	AGENTS.md (hierarchical, auto-detected)
 Permissions Model	Fine-grained allow/deny for tools, files, and domains (Bash, Edit, WebFetch, MCP, etc.)	Sandbox + approval modes (Suggest, Auto Edit, Full Auto); network off by default
 Sandboxing	Workspace-aware with permission rules and safety checks	Stricter sandbox by default; configurable modes and network control
 Extensibility	MCP support, hooks, subagents, skills	MCP support, config profiles, custom commands; open-source CLI surface
 Observability & Debugging	More productized; less of the internals exposed	Highly inspectable (open source); easy to see prompts, tool calls, and logs
 User Experience	Tight UX, opinionated flows, batteries included	CLI-first, more DIY setup, extremely configurable
 Best For	Teams wanting a secure, polished experience out of the box	Power users/teams wanting transparency, customization, and control
 Common Friction	Permission friction; hidden harness behavior	Sandbox/config complexity; approval semantics can be surprising
 Sweet Spot	Long-lived projects, safety-conscious teams, minimal setup	Automation-heavy workflows, custom integrations, experimentation
 Summary	Both are powerful. Claude Code optimizes for a polished, safe experience. Codex CLI optimizes for openness and control.	

https://fieldjournal.ai/blog/codex-cli-vs-claude-code/?utm_source=chatgpt.com

Harnesses must evolve with the models.

- 3 year ago (AutoGen & CrewAI)
- As models improve (e.g., Opus 4.7 context handling), your harness should get simpler.
- Manus rebuilt their harness 5 times

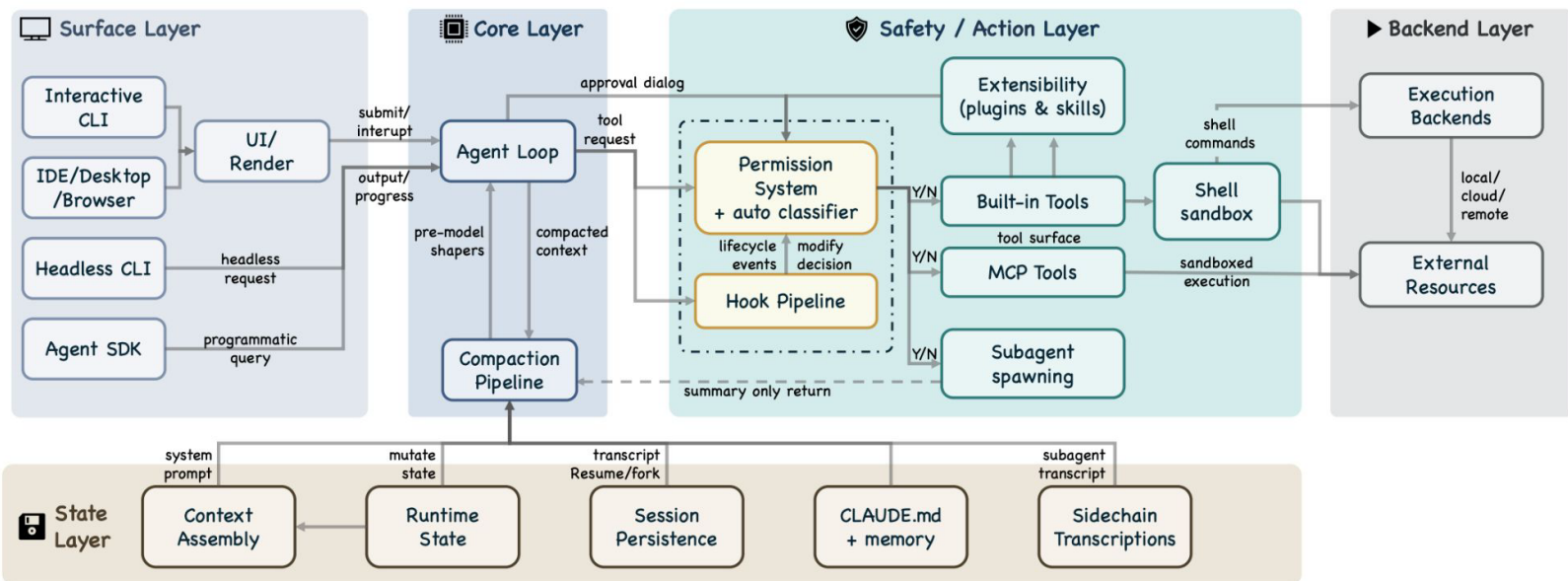


**Noticed the trend towards shorter
system prompts?**

System prompts getting longer!



Claude Code Harness / Architecture



Claude Code Bugs

- default reasoning high -> medium
- bug that accidentally evicted thinking blocks on every turn in session was a change to help with cache optimization
- system prompt change to reduce verbosity which reduced code quality



What model are you using?

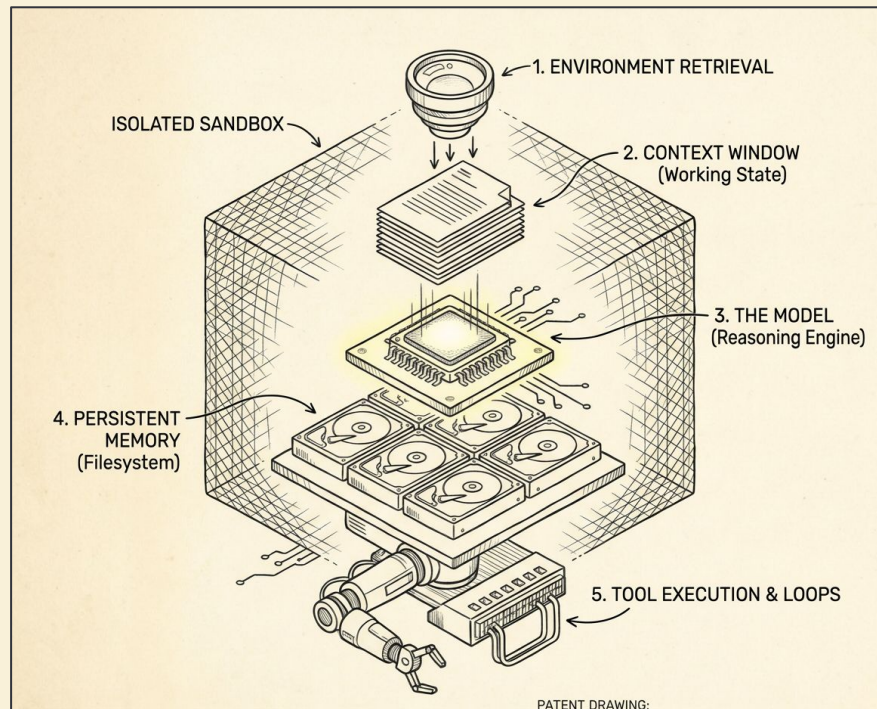
Who owns the harness?

- Stateless chat → easy to switch
- Stateful agents → locked in

**The harness owns your context, memory, and tools.
Pick accordingly.**

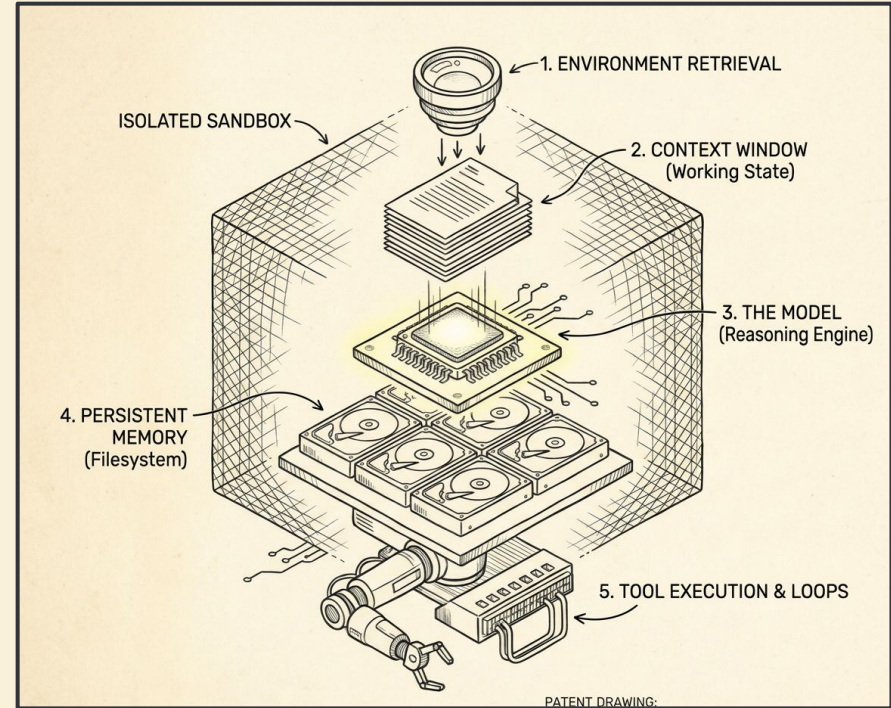
The 5 Levers of Harness Engineering

- The Model
- Retrieval
- Memory & State
- Agentic Loops & Tool Use
- System Architecture



Let's start with how agents find what they need.

- The Model
- Retrieval
- Context & Memory
- Agentic Loops & Tool Use
- System Architecture



The three modes of Agentic Retrieval.



BM25
Keyword-based
retrieval

Language Models
Semantic meaning
with embeddings

Agentic Search
Dynamic using LLM Reasoning

The Baseline: Scaling grep.

- Keyword precision
- Sub-second latency
- Battle-tested



Inverted Indices (BM25) make grep instant.

You don't need a
vector database to
search code.
BM25 is blazing fast.

N_docs	Linear/Grep (s)	Inverted Index	BM25 (s)
1000	3.468	0.005	0.028
3000	10.188	0.014	0.097
6000	20.608	0.025	0.24
9000	30.092	0.061	0.36

State-of-the-Art Coding Agents rely on Lexical Search.

The screenshot shows the Claude Code Docs website. The top navigation bar includes the logo, language selector (English), a search bar, and a star icon. The main navigation menu has links for Getting started, Build with Claude Code, Deployment, Administration, Configuration, Reference (which is underlined), and Agent SDK. On the left, a sidebar lists various sections: Reference, CLI reference, Commands, Environment variables, Tools reference (highlighted with a grey background), Interactive mode, and Checkpointing. The main content area displays a list of tools. A pink dashed box highlights three tools: 'Glob' (Finds files based on pattern matching), 'Grep' (Searches for patterns in file contents), and 'ListMcpResourcesToo' (Lists resources exposed by connected MCP servers). Other visible tools include 'ExitPlanMode' (Presents a plan for approval and exits plan mode) and 'ExitWorktree' (Exits a worktree session and returns to the original directory). The text 'Not available to subagents' appears next to the 'ExitWorktree' tool.

Tool	Description
ExitPlanMode	Presents a plan for approval and exits plan mode
ExitWorktree	Exits a worktree session and returns to the original directory. Not available to subagents
Glob	Finds files based on pattern matching
Grep	Searches for patterns in file contents
ListMcpResourcesToo	Lists resources exposed by connected <u>MCP servers</u>

Where Lexical breaks down: The Synonym Gap.

Synonym Gap (Vocabulary Mismatch)

Query: "physician salary cap policy"

Doc A (relevant): "Doctor compensation limits for hospitals..." ●

Doc B (distractor): "Company salary cap policy for managers..." ✗

BM25 Overlaps on "salary", "cap", "policy" → often ranks Doc B above Doc A.

Why: No shared token for physician↔doctor.

Aliases & Abbreviations

Query: "International Business Machines layoffs 2024"

Doc A (relevant): "IBM announced workforce reductions in 2024..." ●

Doc B (distractor): "International business trends show slower layoffs..." ✗

BM25 Matches literal words → Doc B can outrank Doc A.

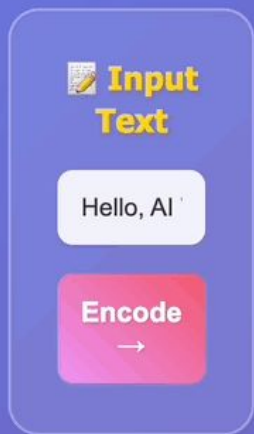
Why: Misses IBM ≡ International Business Machines.

Takeaway:

- If you have keyword-heavy queries and need sub-second response → text search might be sufficient

Embeddings solve for meaning.

Text → Encoder → Embedding



NEURAL ENCODER



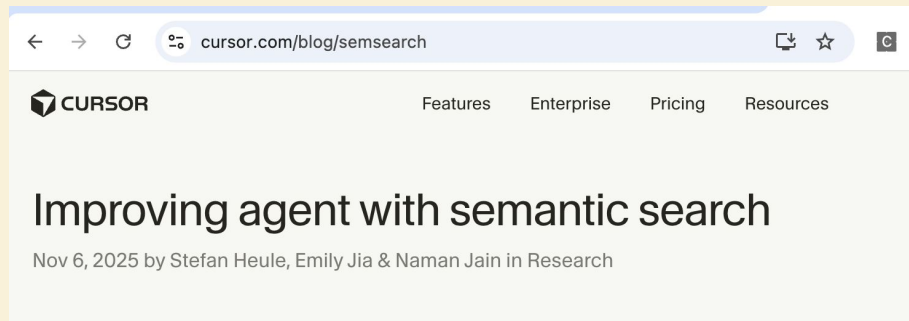
 **Embedding Vector**

[0] 0.9169	[1] 0.4592	[2] 0.0133	[3] 0.2671
[4] -0.8128	[5] 0.3155	[6] 0.0590	[7] 0.9169
[8] 0.2248	[9] 0.9617	[10] 0.6653	[11] -0.4390
[12] 0.1665	[13] -0.1237	[14] -0.4086	[15] 0.2353

Semantic search is still required for massive codebases.

● Cursor

- Achieving on average 12.5% higher accuracy
- Producing code changes that are more likely to be retained in codebases.

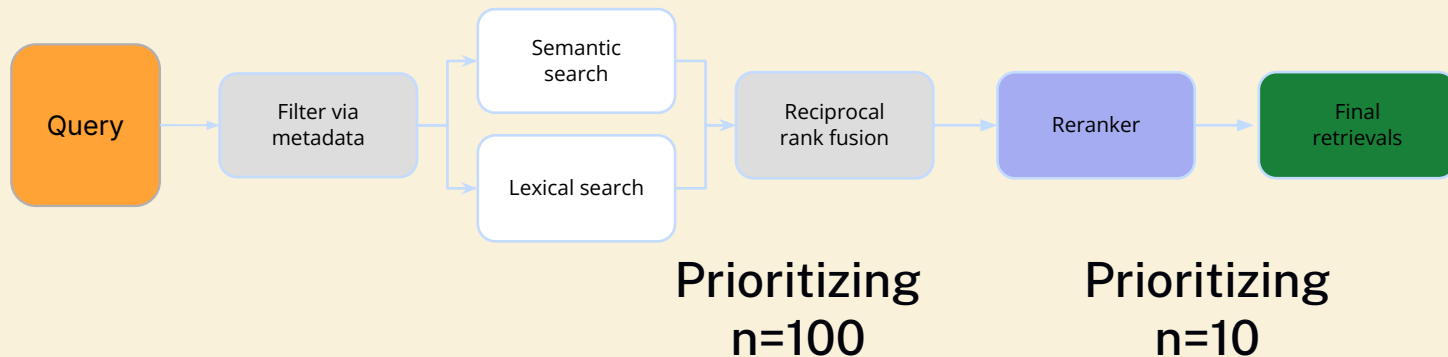


<https://cursor.com/blog/semsearch>

The Status Quo: Hybrid Search.

Search Performance

Dataset	BM25	Dense	Hybrid
HotpotQA	0.61	0.29	0.64
Quora	0.62	0.82	0.85
DBPedia	0.32	0.28	0.37



Search Strategy Comparison

Watch how different approaches explore the solution space

Traditional RAG



Ready to search...

0

Queries Made

0ms

Time Taken

Agentic RAG



Ready to explore...

0

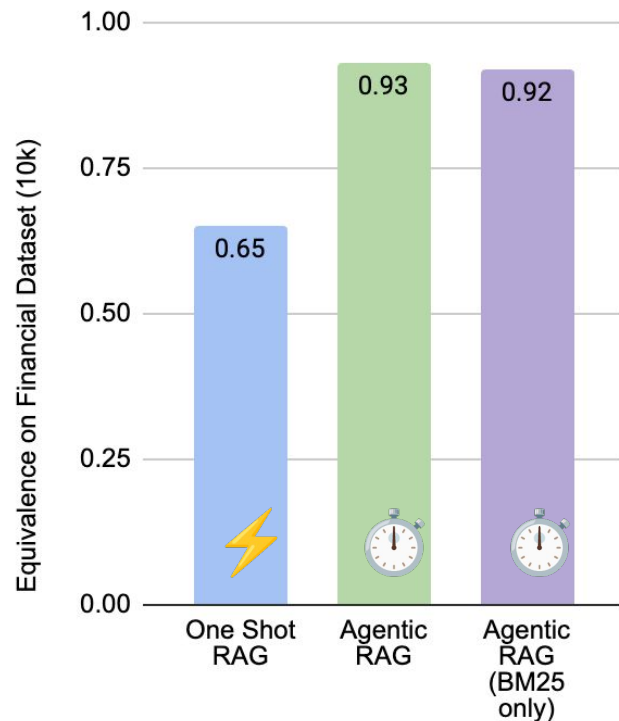
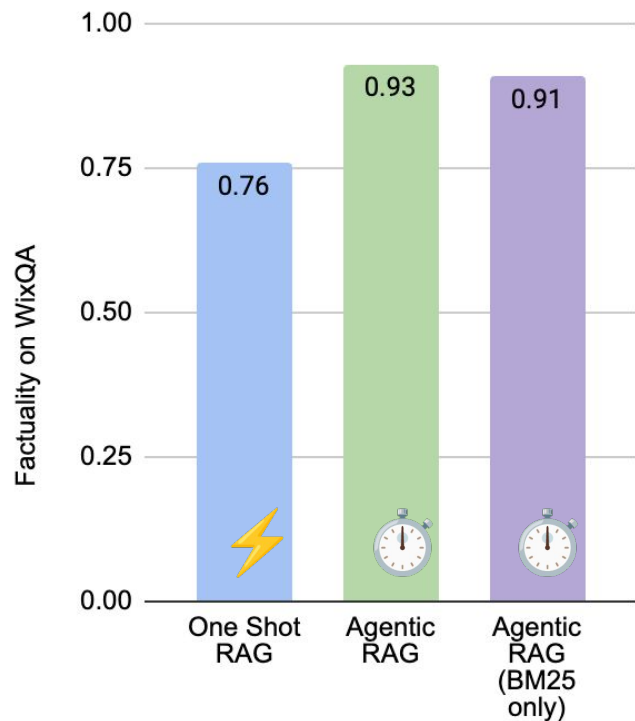
Queries Made

0ms

Time Taken

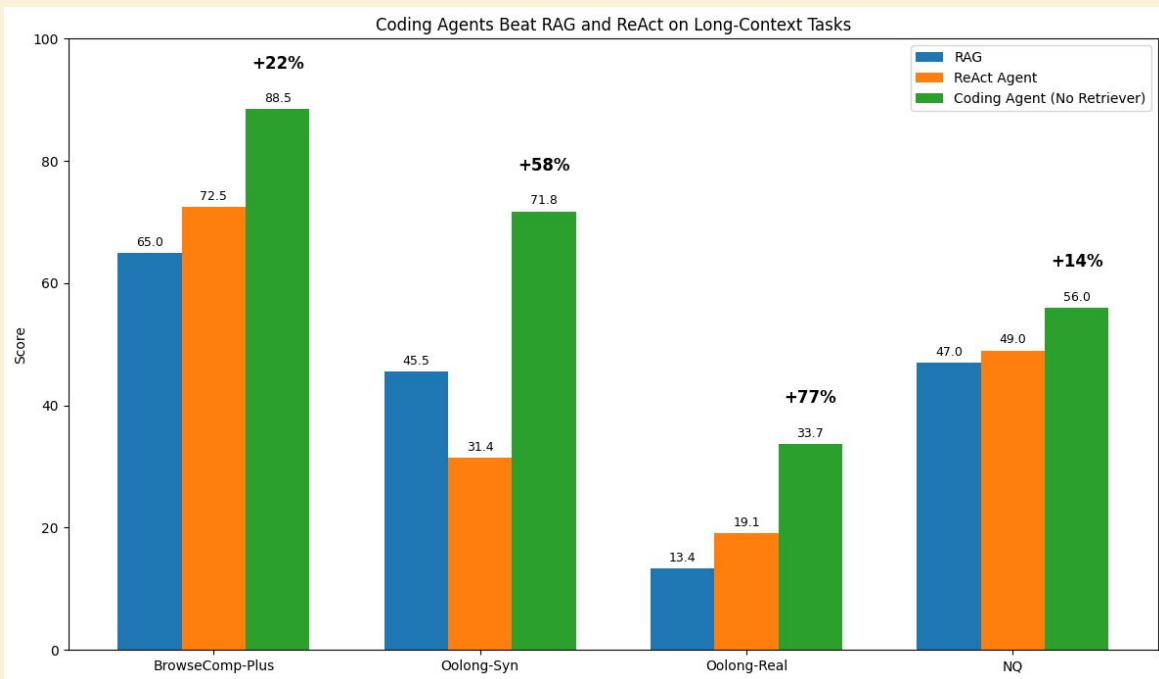
Agentic Search tradeoffs?

Agentic Search trades latency for massive accuracy.



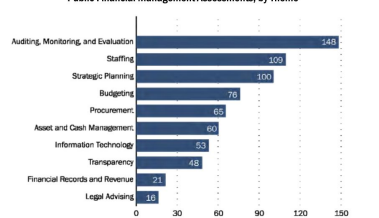
Coding agents are really good at long Context

- Coding agents with file access are very good



Don't chunk if you don't have to: Whole files > RAG.

Figure 1 - Number of Weaknesses Found across Ernst & Young's and KPMG's Public Financial Management Assessments, by Theme



Source: SIGAR analysis of Ernst & Young's and KPMG's public financial management assessments.

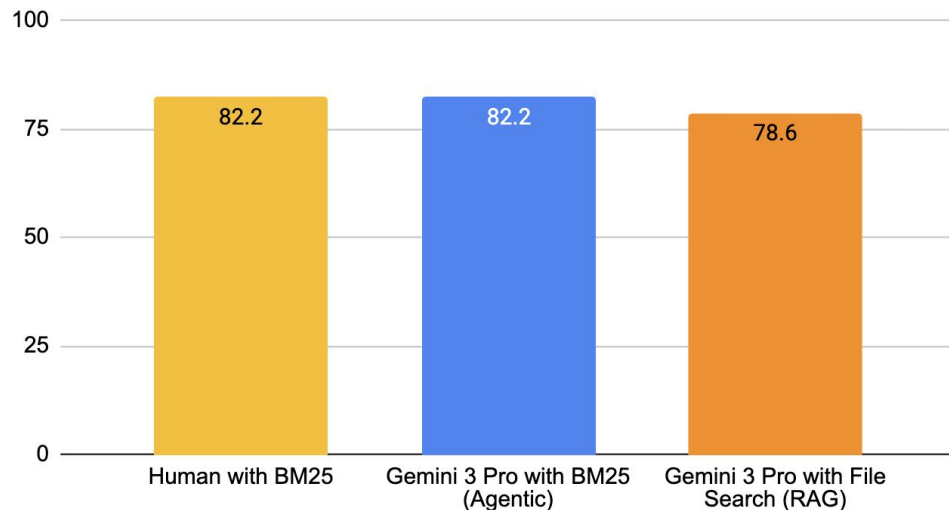
USAID's Risk Reviews Found that the Afghan Ministries Were Unable to Manage Donor Funds and Lacked the Capability to Combat Corruption

USAID/Afghanistan used Ernst & Young's and KPMG's public financial management assessments to complete risk reviews of the seven ministries that had active or planned direct assistance programs as of August 2013.²⁷ In its risk reviews, USAID/Afghanistan identified 104 major risks associated with relying on the ministries to manage donors' direct assistance funds. Risks for individual ministries ranged from 11 for the Ministry of Agriculture, Irrigation, and Livestock to 26 for the Ministry of Finance. Although the risks varied by ministry, they generally identified problems with the ministries' management and governance structures, financial management and accounting systems, personnel procedures, procurement and purchasing systems, and program management. For example, USAID/Afghanistan found that the Ministry of Public Health was at risk of "concealing vital monitoring and evaluation information" and "misappropriation of cash arising from payment of salaries in cash." In its review of the Ministry of Mines and Petroleum, USAID/Afghanistan noted that "waste, fraud, and abuse may go undetected" and identified the risk of "paying higher prices for commodities and services to finance kickbacks and bribes." Each risk review noted similar types of weaknesses. The mission rated 99 of the 104 risks, or about 95 percent, as "critical" or "high."²⁸

²⁷ The Independent Administrative Reform and Civil Service Commission has an active program, but USAID/Afghanistan had not completed a risk review of the commission or planned to do one, as of August 2013. The Supreme Court and Ministry of Justice have planned programs, and the contractors are still conducting public financial management assessments of these two ministries.

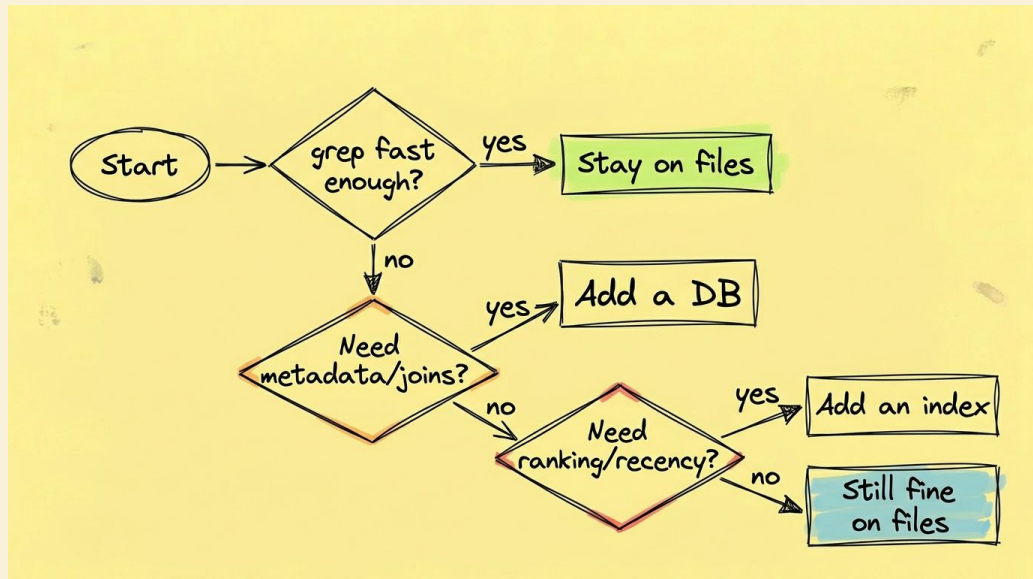
²⁸ USAID/Afghanistan assigned risk ratings based on the probability of an "adverse event associated with [a] risk" occurring, ranging from remote to frequent, combined with the adverse event's potential impact, ranging from negligible to catastrophic. Appendix V includes a matrix of how USAID/Afghanistan assigns risk ratings.

Multimodal Agentic Document QA (MADQA) Benchmark



So when should you move to a database

Files stay the source of truth. Index on top when query cost bites.

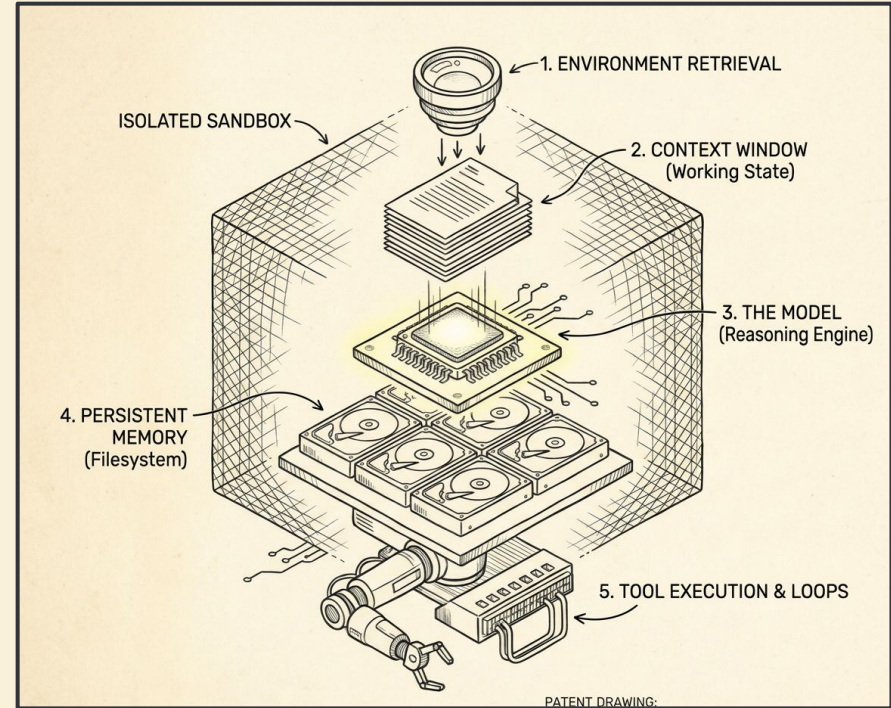


Design rules for you

- **Lexical for Small Set of Files:** BM25 / grep is your baseline.
- **Add Semantics for Speed:** Only if you suffer from vocabulary mismatch.
- **Loop it if Accuracy is Key:** Ultimately, put the retrieval inside an iterative agentic loop.

Let's start with how agents find what they need.

- The Model
- Retrieval
- Context & Memory
- Agentic Loops & Tool Use
- System Architecture



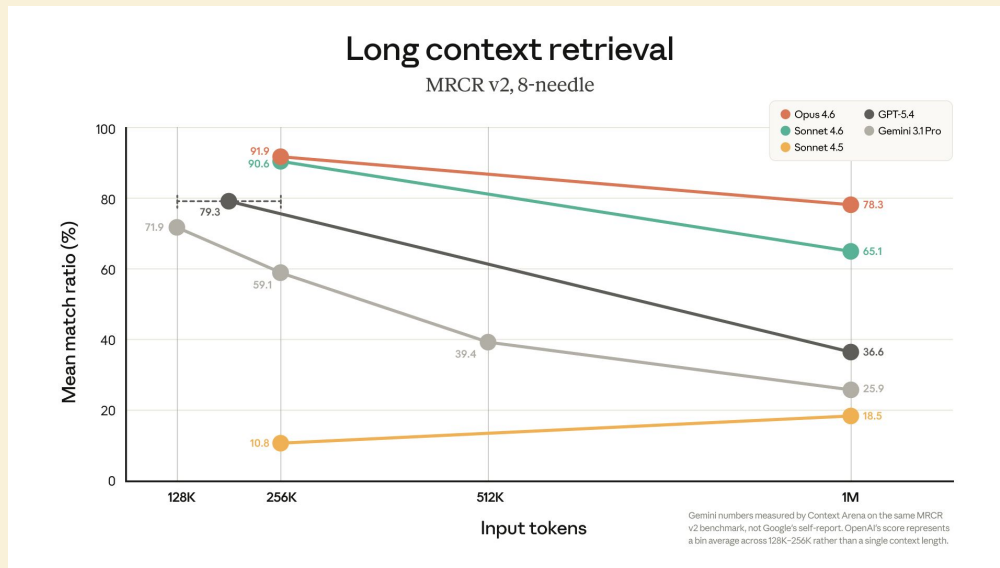
Memory & State

**Agents usually fail from losing the thread,
not from hitting the token limit**

Are you excited about 10M Context Windows?

1M Context Windows are a trap.

- 500 page multimodal PDF
- 20k rows of data
- 100k lines / 5 MB



Models degrade when you stuff them full.

Verbose tool traces crowd out the goal.

```
$ npm install -g expo-cli
npm WARN deprecated urix@0.1.0: Please see https://github.com/lydell/urix#depre
ated
npm WARN deprecated har-validator@5.1.3: this library is no longer supported
npm WARN deprecated chokidar@2.1.8: Chokidar 2 will break on node v14+. Upgrade
to chokidar 3 with 15x less dependencies.
npm WARN deprecated chokidar@2.1.8: Chokidar 2 will break on node v14+. Upgrade
to chokidar 3 with 15x less dependencies.
npm WARN deprecated resolve-url@0.2.1: https://github.com/lydell/resolve-url#depre
cated
npm WARN deprecated uuid@3.0.0: Please upgrade to version 7 or higher. Older v
ersions may use Math.random() in certain circumstances, which is known to be pro
blematic. See https://v8.dev/blog/math-random for details.
npm WARN deprecated querystring@0.2.0: The querystring API is considered Legacy.
new code should use the URLSearchParams API instead.
npm WARN deprecated uuid@3.4.0: Please upgrade to version 7 or higher. Older v
ersions may use Math.random() in certain circumstances, which is known to be pro
blematic. See https://v8.dev/blog/math-random for details.
npm WARN deprecated uuid@3.4.0: Please upgrade to version 7 or higher. Older v
ersions may use Math.random() in certain circumstances, which is known to be pro
blematic. See https://v8.dev/blog/math-random for details.
npm WARN deprecated uuid@3.4.0: Please upgrade to version 7 or higher. Older v
ersions may use Math.random() in certain circumstances, which is known to be pro
blematic. See https://v8.dev/blog/math-random for details.
npm WARN deprecated request@2.88.2: request has been deprecated, see https://git
hub.com/request/request/issues/3142
npm WARN deprecated svgo@1.3.2: This SVGO version is no longer supported. Upgrad
e to v2.x.x.
npm WARN deprecated graphql-tools@8.0.0: This package has been deprecated and no
w it only exports makeExecutableSchema. And it will no longer receive updates. We
recommend you to migrate to scoped packages such as @graphql-tools/schema, @gr
aphql-tools/utils and etc. Check out https://www.graphql-tools.com to learn wh
at package you should use instead
```

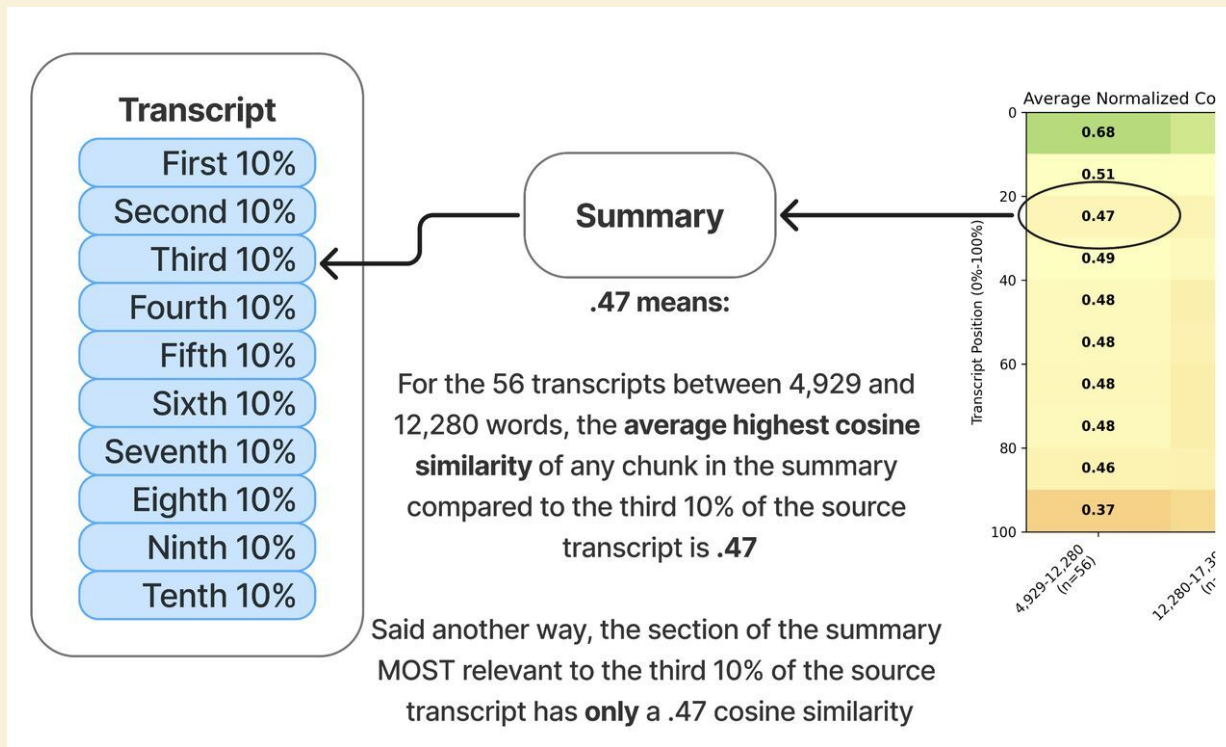
Claude Opus 4.6 Struggles Beyond 200K Tokens in Real Coding Sessions

Last updated 14 hours ago

Vercel developer Melkey shared a screenshot showing Claude Code degrade sharply at 29% context usage, calling the 0-15% range the only reliable sweet spot. Other users report similar issues like repetitive suggestions and lost memory after 150-250k tokens, despite Anthropic's strong benchmarks on tests like MRCR v2. Workarounds include early auto-compaction hacks or sticking to 200k limits, as devs weigh alternatives like GPT 5.4 or open-source models.

When 100s of lines of warning push the real goal out of the agent's context window.

Key facts disappear inside long model inputs



Three layers of Memory

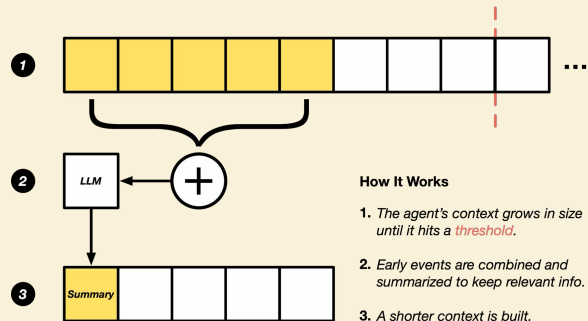
- **Layer 1 - Active Context:** What is in the prompt right now.
- **Layer 2 - Working State:** Plans, TODOs, scratchpads.
- **Layer 3 - Durable Memory:** Skills and reusable workflows.

Layer 1: Fixing Active Context



Reset:

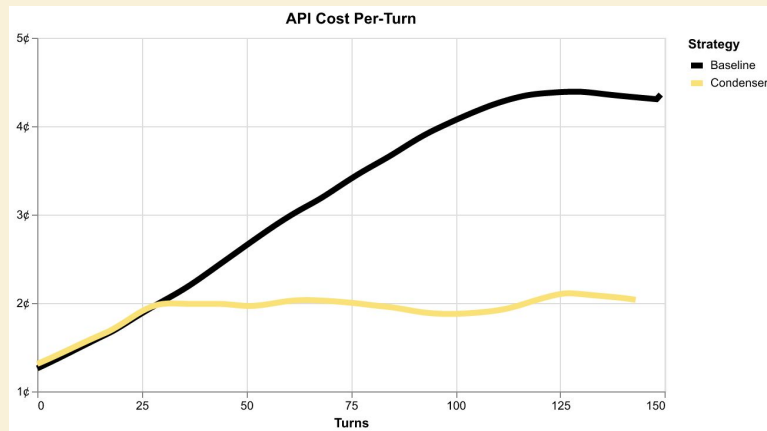
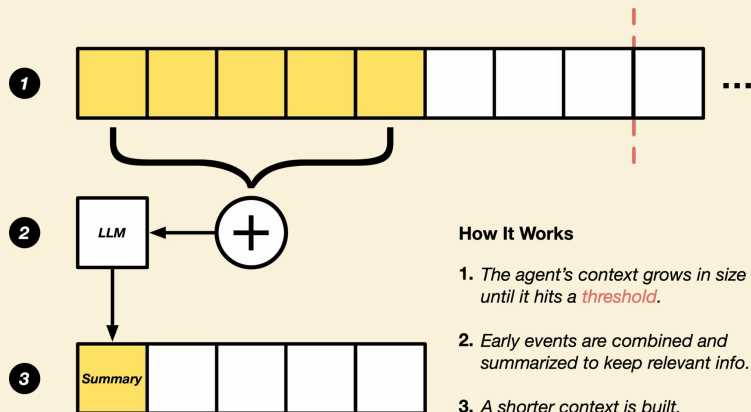
- Clear the window entirely.
- Refill it with only the original instructions and critical artifacts.



Compacting:

- Summarize older turns but keep recent turns intact

Layer 1: Compacting from OpenHands



Up to 2x per-turn API cost reduction

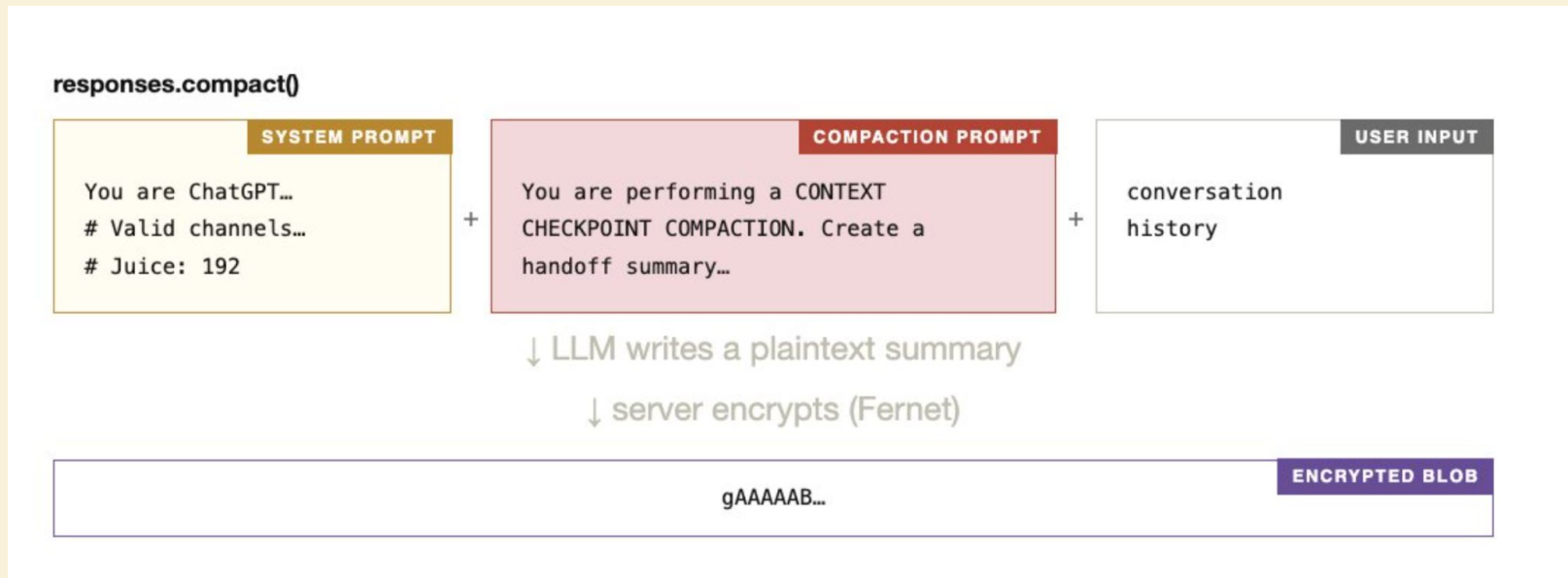


Consistent response times in long sessions



Equivalent (or better!) performance on software engineering tasks

Layer 1: What's your compaction approach?



Getting the most of a 1M Context Windows

Techniques from Anthropic on managing your context window

Correcting



context = reads + 2 failed attempts + 2 corrections + the fix

Rewinding



context = reads + one informed prompt + the fix

continue

rewind

/clear

/compact

subagent

Layer 2: Working State (The Golden Rule)

Files make better memory than chat.

Write your plan to a .md file in the workspace rather than holding it in prompt memory.

Task Breakdown

Phase 1 — Project Setup

- ☐ Initialise Node.js project (`npm init`)
- ☐ Install dependencies: `express`, `prisma`, `jsonwebtoken`, `bcrypt`, `dotenv`
- ☐ Configure `.env` with `DATABASE_URL`, `JWT_SECRET`, `JWT_REFRESH_SECRET`
- ☐ Set up Prisma schema with `User` model

Phase 2 — Database Layer

- ☐ Define `User` model in `schema.prisma`
 - Fields: `id`, `email`, `passwordHash`, `createdAt`, `refreshToken`
- ☐ Run initial migration: `npx prisma migrate dev`
- ☐ Create `db.ts` Prisma client singleton

Phase 3 — Auth Logic

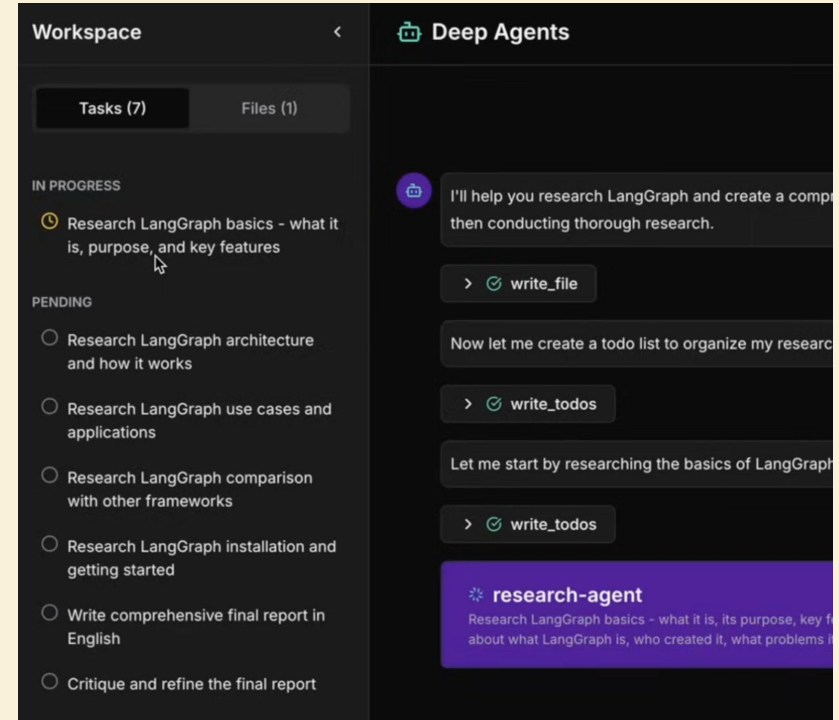
- ☐ `auth/hash.ts` — `hashPassword()` and `verifyPassword()` helpers
- ☐ `auth/tokens.ts` — `generateAccessToken()`, `generateRefreshToken()`, `verifyToken()`
- ☐ Token expiry: access = 15 min, refresh = 7 days

Phase 4 — API Routes

- ☐ `POST /auth/register` — validate input, hash password, create user
- ☐ `POST /auth/login` — verify credentials, return token pair
- ☐ `POST /auth/refresh` — validate refresh token, issue new access token
- ☐ `POST /auth/logout` — invalidate refresh token in DB
- ☐ `GET /me` — protected route returning current user profile

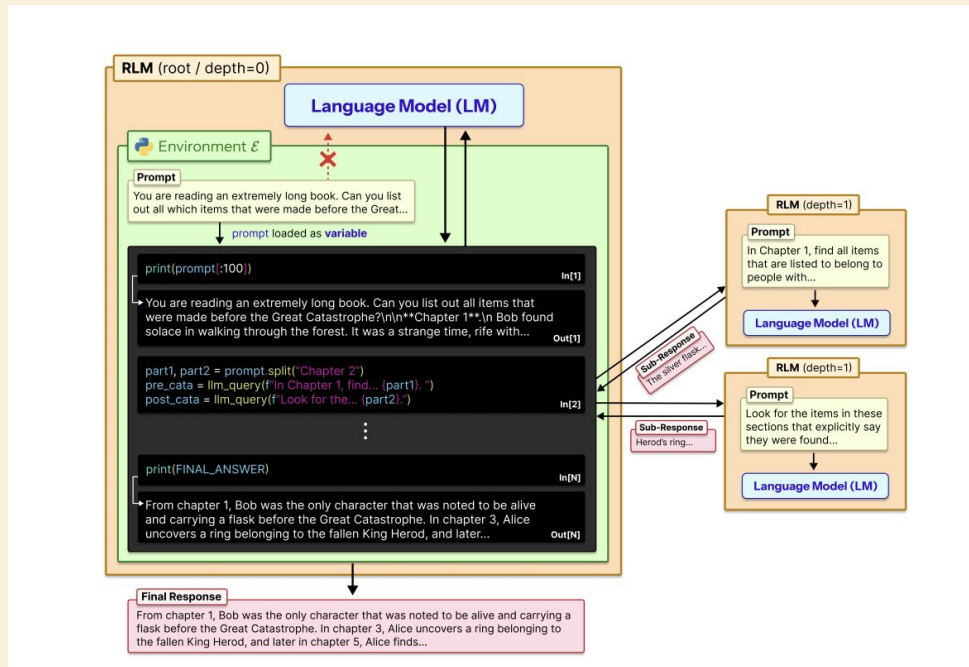
Deep Agents rely on external plans.

LangChain's Deep Agents
uses a `write_todos` tool to write
out plans for agent tasks



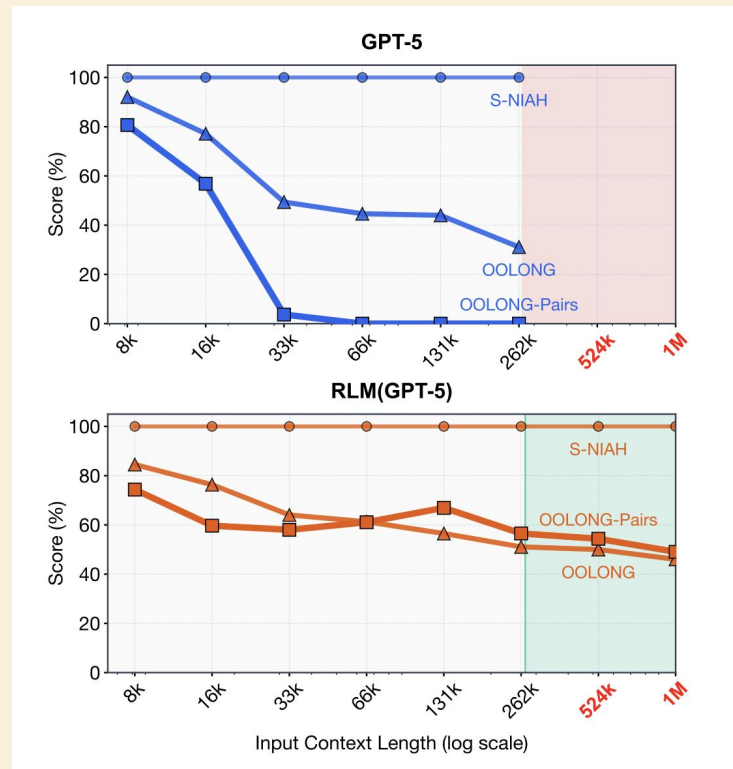
Extreme Layer 2: Recursive Language Models (RLM)

RLMs bypass token limits entirely by using a persistent Python REPL to manage their state and call sub-LLMs.

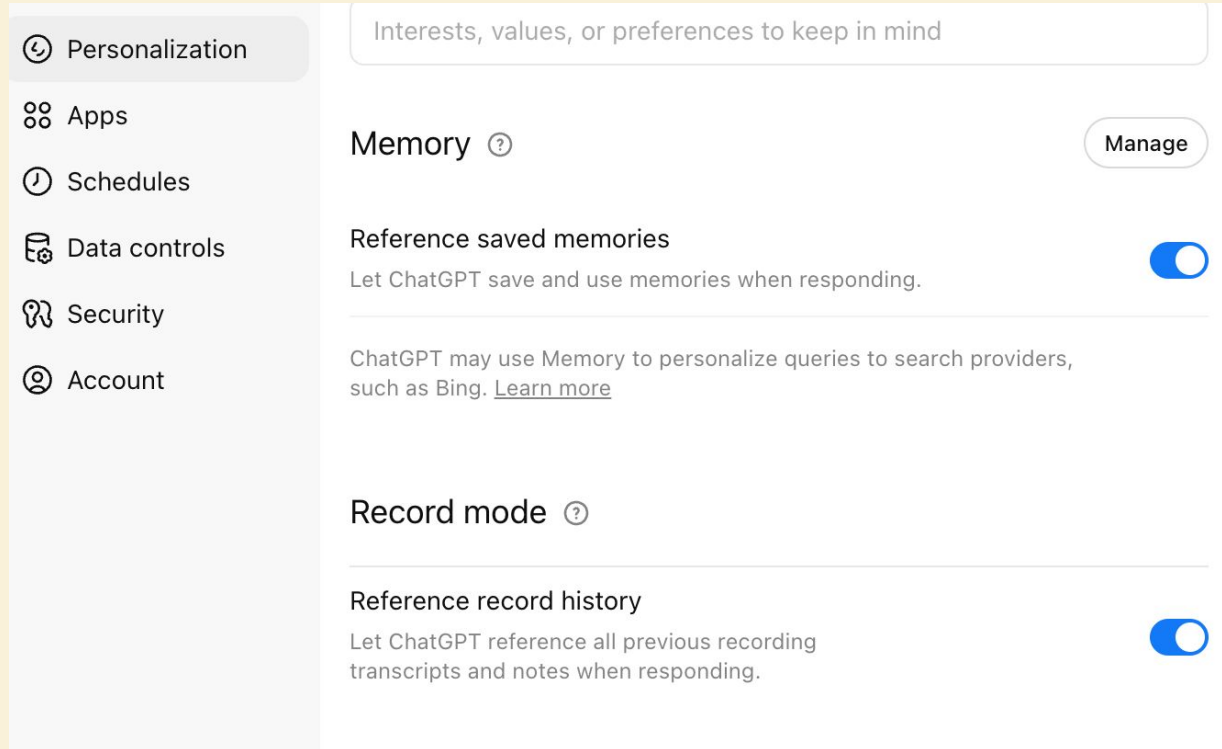


RLMs maintain accuracy at 1M tokens.

Standard GPT-5 failed entirely;
RLM-GPT-5 hit 91% accuracy



Layer 3: Durable Memory



The screenshot displays the 'Personalization' settings page in a web browser. On the left, a sidebar lists various settings categories: Personalization (selected), Apps, Schedules, Data controls, Security, and Account. The main content area is titled 'Interests, values, or preferences to keep in mind'. Below this, the 'Memory' section is visible, featuring a 'Manage' button and a toggle for 'Reference saved memories', which is currently turned on. A descriptive text explains that ChatGPT may use memory to personalize queries to search providers like Bing, with a link to 'Learn more'. The 'Record mode' section follows, also with a toggle for 'Reference record history' turned on, explaining that ChatGPT will reference previous recording transcripts and notes.

Personalization

Apps

Schedules

Data controls

Security

Account

Interests, values, or preferences to keep in mind

Memory ?

Manage

Reference saved memories

Let ChatGPT save and use memories when responding.

ChatGPT may use Memory to personalize queries to search providers, such as Bing. [Learn more](#)

Record mode ?

Reference record history

Let ChatGPT reference all previous recording transcripts and notes when responding.

The Reality: Unintended Distractions



Who uses an Agents.md file?

Durable Memory with Agents.md

AGENTS.md

A simple, open format for guiding coding agents adopted by more than 60,000 open-source projects and agent frameworks including:



Codex
from OpenAI



Amp



Jules
from Google



Cursor



Factory



VS Code



Devin
from Cognition



Gemini CLI
from Google



Copilot
from GitHub

Sample AGENTS.md file

Dev environment tips

- Use ``pnpm dlx turbo run where <project_name>`` to jump to a package install
- Run ``pnpm install --filter <project_name>`` to add the package to your v
- Use ``pnpm create vite@latest <project_name> -- --template react-ts`` to
- Check the name field inside each package's package.json to confirm the

Testing instructions

- Find the CI plan in the `.github/workflows` folder.
- Run ``pnpm turbo run test --filter <project_name>`` to run every check de
- From the package root you can just call ``pnpm test``. The commit should
- To focus on one step, add the Vitest pattern: ``pnpm vitest run -t "<tes``
- Fix any test or type errors until the whole suite is green.
- After moving files or changing imports, run ``pnpm lint --filter <projec``
- Add or update tests for the code you change, even if nobody asked.

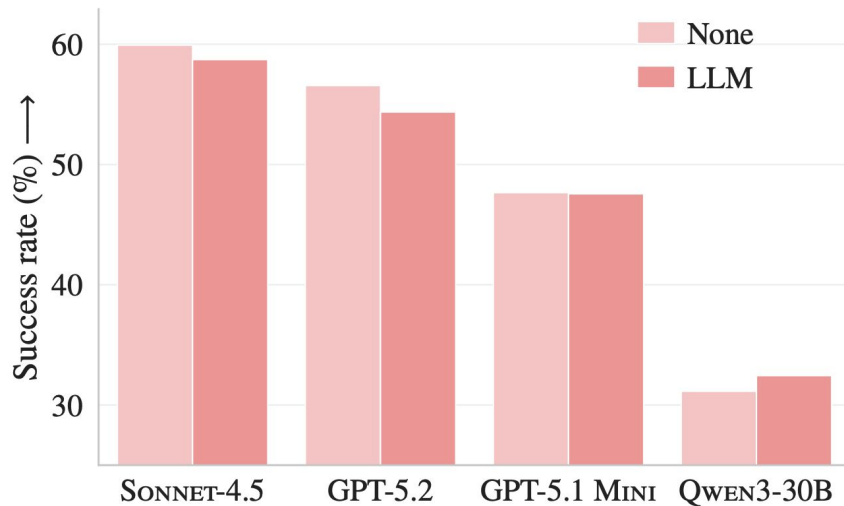
PR instructions

- Title format: [`<project_name>`] `<Title>`
- Always run ``pnpm lint`` and ``pnpm test`` before committing.

Don't overload this file, it's part of every prompt

Auto-generated AGENTS.md files hurt performance

Across multiple coding agents and LLMs, we find that context files tend to reduce task success rates compared to providing no repository context, while also increasing inference cost by over 20%.



The rule of thumb: "Minimize Load-Bearing Memory"

And so the thing that you want is do the minimal possible thing in order to get the model on track. And so if you delete your `claude.md` and then, you know, the model is getting off track, it does the wrong thing, that's when you kind of add back a little bit at a time. And what you're probably going to find is with every model you have to add less and less.



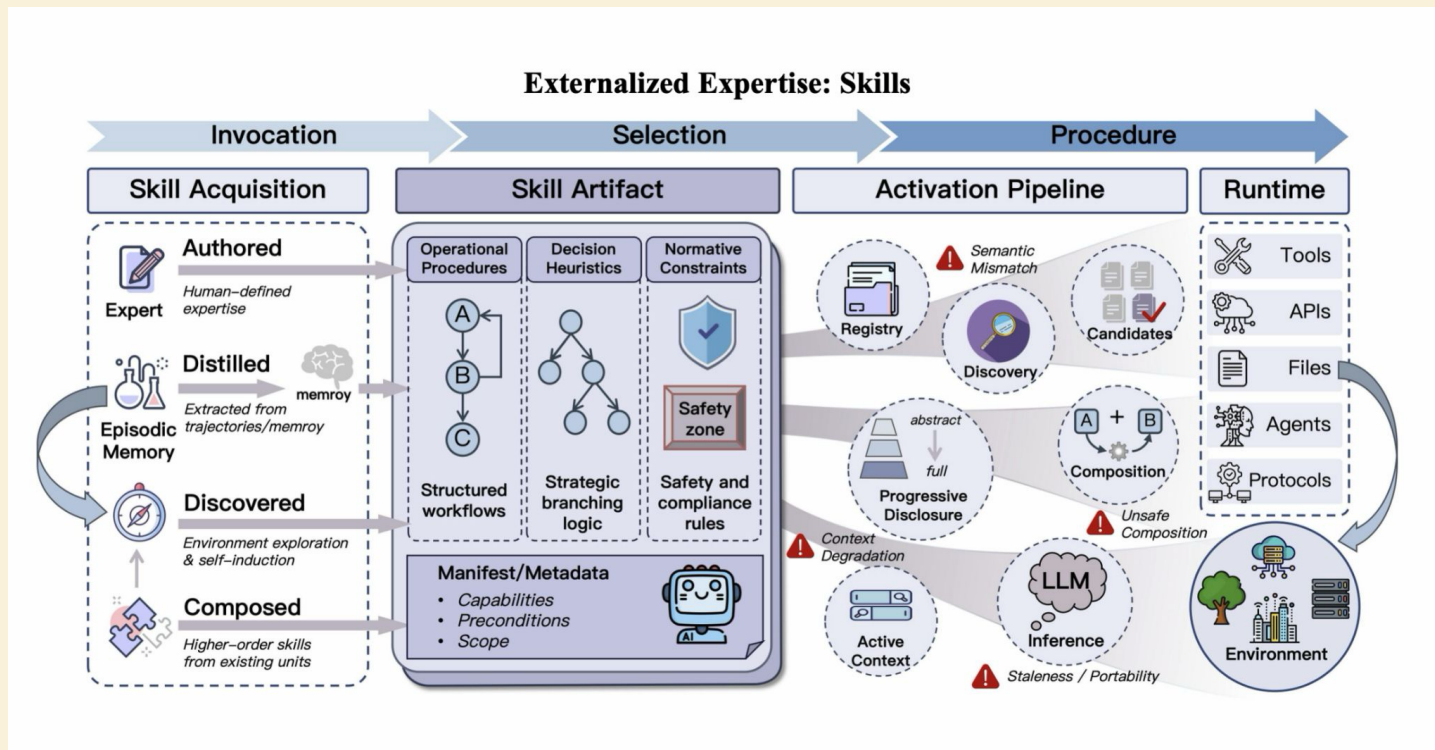
Skills are the new standard for Durable Memory.

A skill isn't just a prompt.

It's a combo of a trigger, a reference manual, and a script, e.g, a skill for querying your internal company metrics API



Skills as Externalized Expertise



Skills can replace Code

NEW

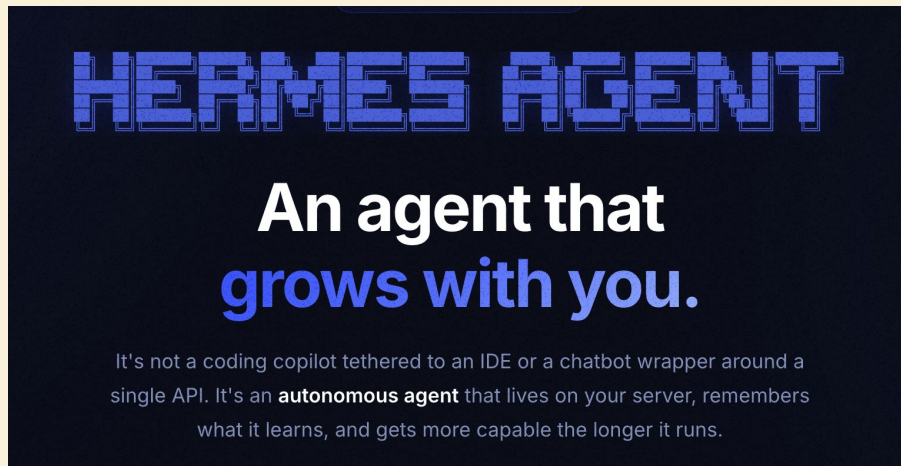
Implementation Overview

- UI/UX for worktree creation and management
- Agent Loop and tool call worktree scoping
- Worktree setup script execution
- Best of N judging
- Harness changes and system reminders
- Worktree clean up
- ~200 LoC skill

151 files +2,142 -15,328 Updated 5d ago

Building a learning loop

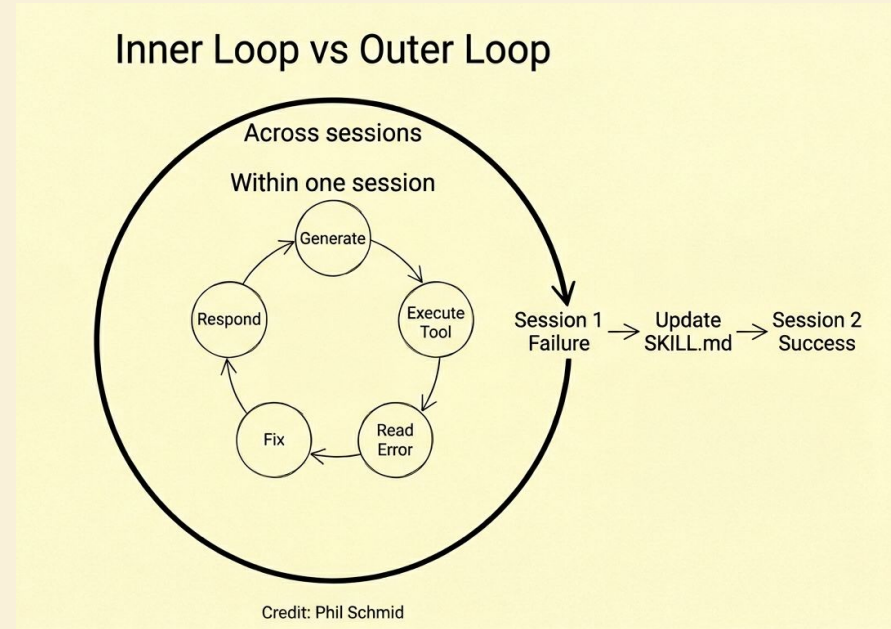
1. Task attempt: Run complex task
2. Periodic nudge: "What would you do differently?"
3. Agent writes skill: Saves to skills/.md
4. Self-improves: Edits own skill on next failure



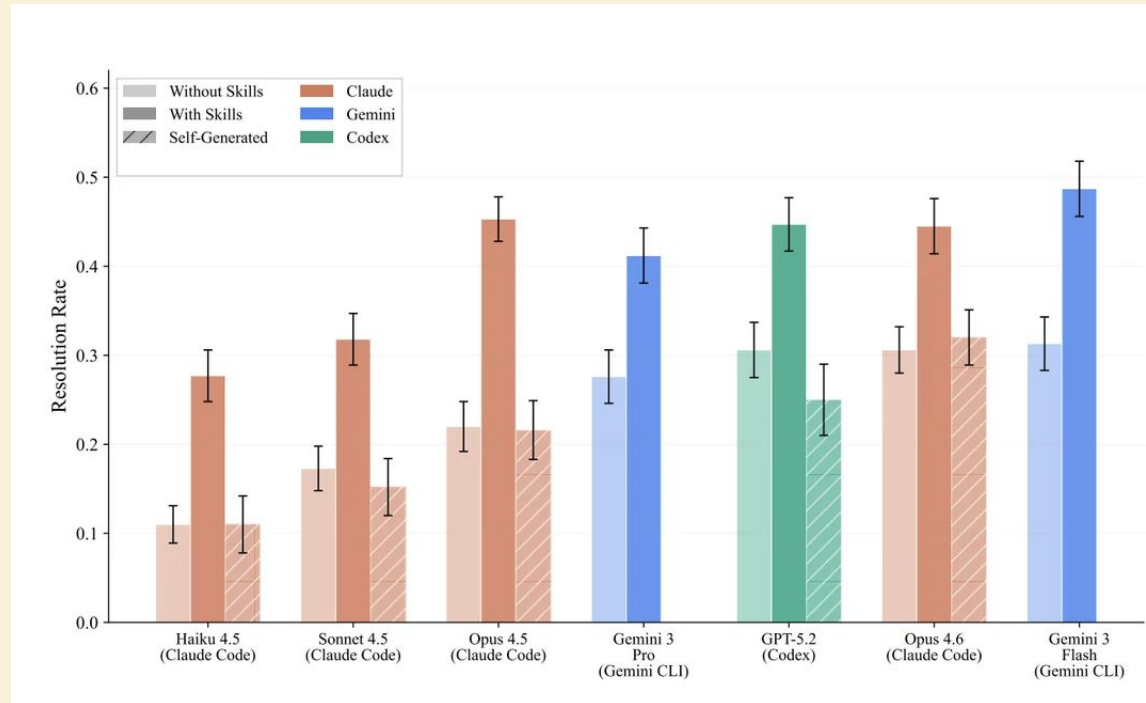
Continual learning outer loop with skills

Inner loop finishes the task.

Outer loop makes the agent smarter.



Warning: 16% of skills actually reduce performance.



You must evaluate your skills.

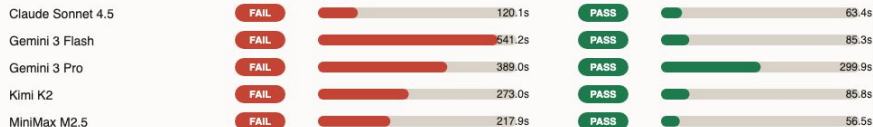
Model breakdown by task

Each task gets its own section, with pass/fail and runtime bars for no-skill vs improved-skill.

Dependency audit

No skill

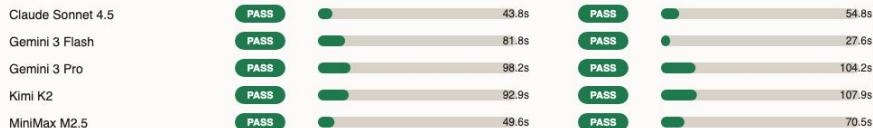
Improved skill



SEC financial report

No skill

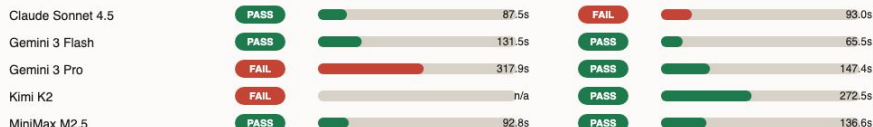
Improved skill



Sales pivot analysis

No skill

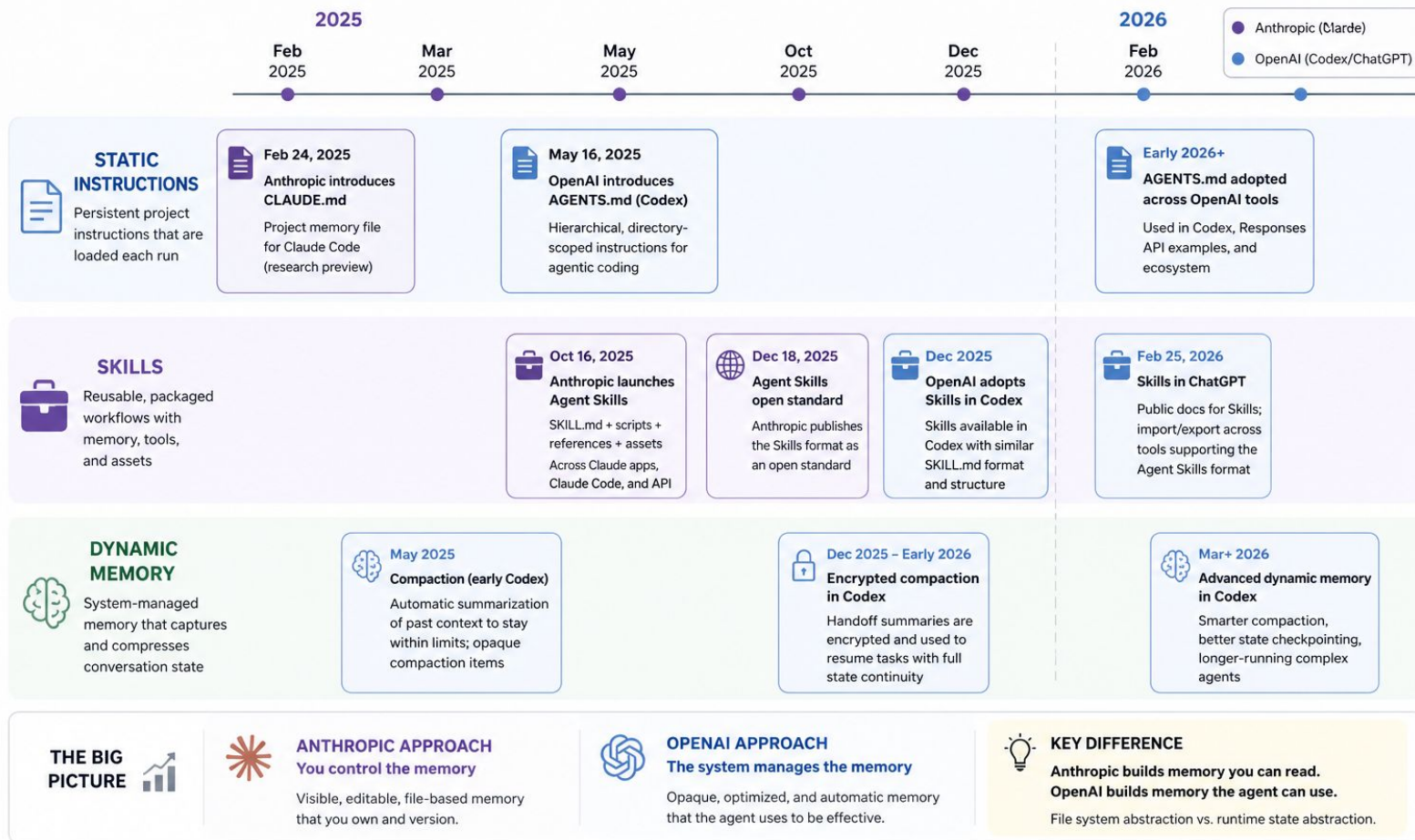
Improved skill



Longer bars mean longer runtime, scaled to the slowest run in the full matrix.

AGENT MEMORY EVOLUTION TIMELINE: ANTHROPIC vs OPENAI

When key memory features were introduced and adopted in coding agent harnesses



Sources: Anthropic announcements, OpenAI blog & docs, public timelines, and product documentation (Feb 2025 – Mar 2026)

Claude Code

What gets locked in:

- Conversation compaction and context assembly
- CLAUDE.md / settings / project memory
- Tool search and MCP loading strategy
- Subagent definitions and delegation behavior
- Permission rules, hooks, and policy surfaces



Memory as Lock-in

Closed Harness

- Memory lives in provider API
- Compaction is encrypted
- Switch provider → lose history

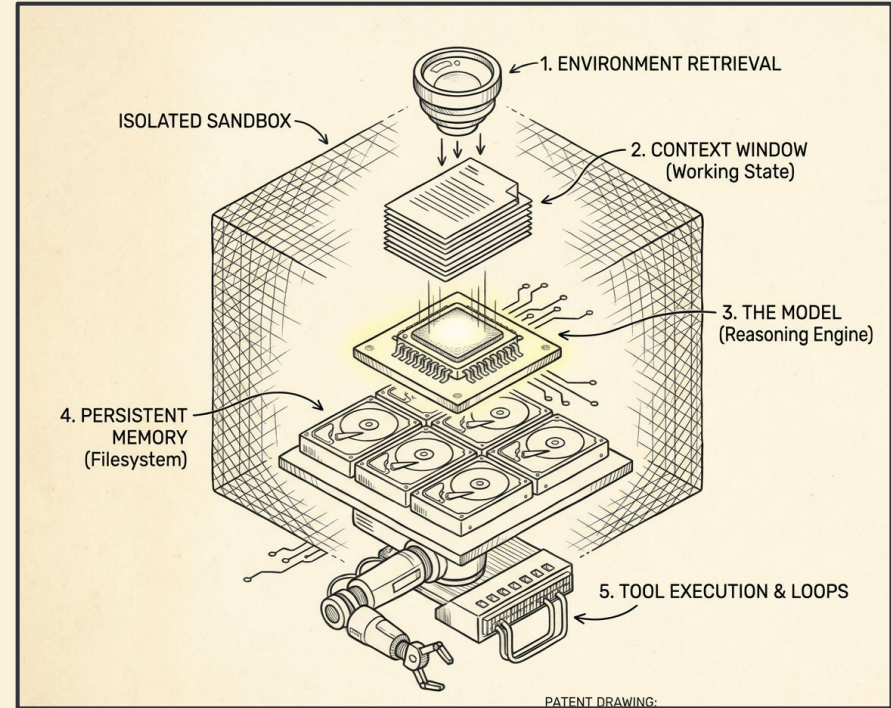
Open Harness

- Memory lives in your files
- Compaction is your code
- Switch provider → memory stays

Lock the harness → lose the memory → lose your product.

Let's start with how agents find what they need.

- The Model
- Retrieval
- Context & Memory
- Agentic Loops & Tool Use
- System Architecture



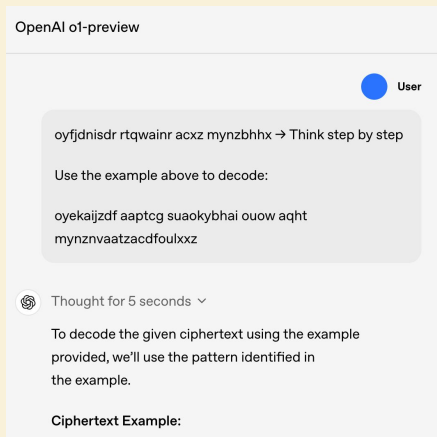
Agentic Loops and Tool Use

Better tool loops beat better prompts.

Engineering the Loop

1. The Baseline: The "Ralph Wiggum" loop (and why it fails)
2. Cognitive Discipline: Forcing the model to think via JSON schemas
3. Environmental Discipline: Using tests & CI to physically block bad loops
4. Safety & Friction: Sandboxing autonomous actions

We no longer rely on single-shot execution.



O1 - Sept 2024

We no longer rely on single-shot execution.

Opus 4.7 Tools:

ask_user_input_v0	recipe_display_v0
bash_tool	recommend_claude_apps
conversation_search	search_mcp_registry
create_file	str_replace
fetch_sports_data	suggest_connectors
image_search	view
message_compose_v1	weather_fetch
places_map_display_v0	web_fetch
places_search	web_search
present_files	tool_search
recent_chats	visualize:read_me
	visualize:show_widget

Have you retried the same thing over and over again?

The Default: The "Ralph Wiggum" Agent

Try command
Get failure
Retry without diagnosis
Repeat until stopped by the harness

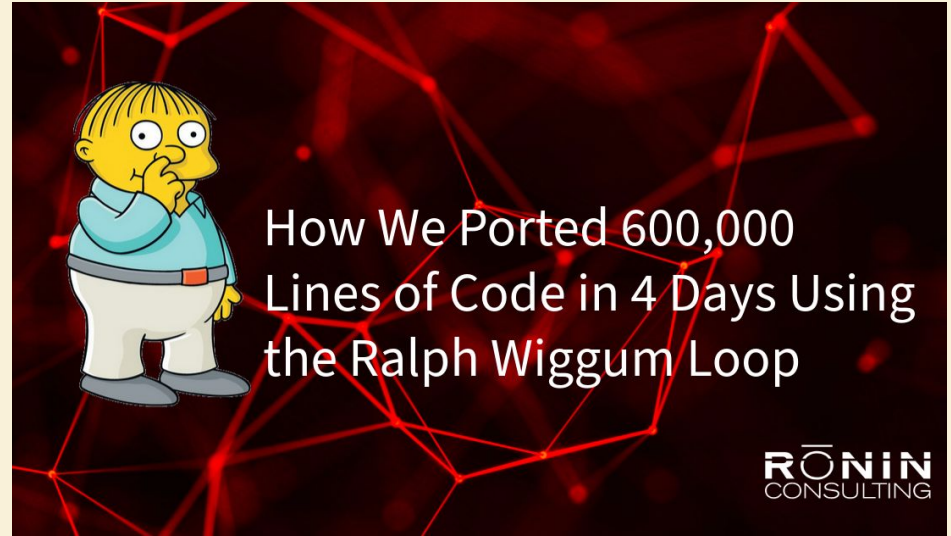


Ralph can work

If the plan is perfect and
nothing breaks, Ralph can
build things.

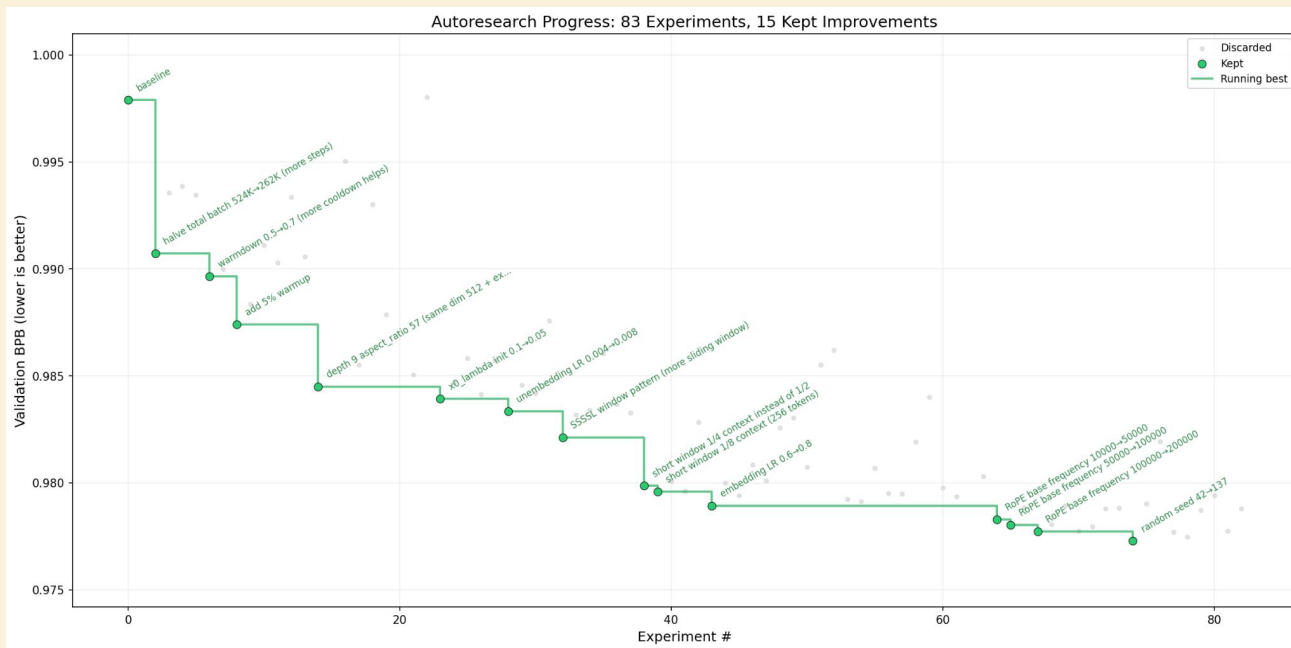
But we can do better.

C Compiler



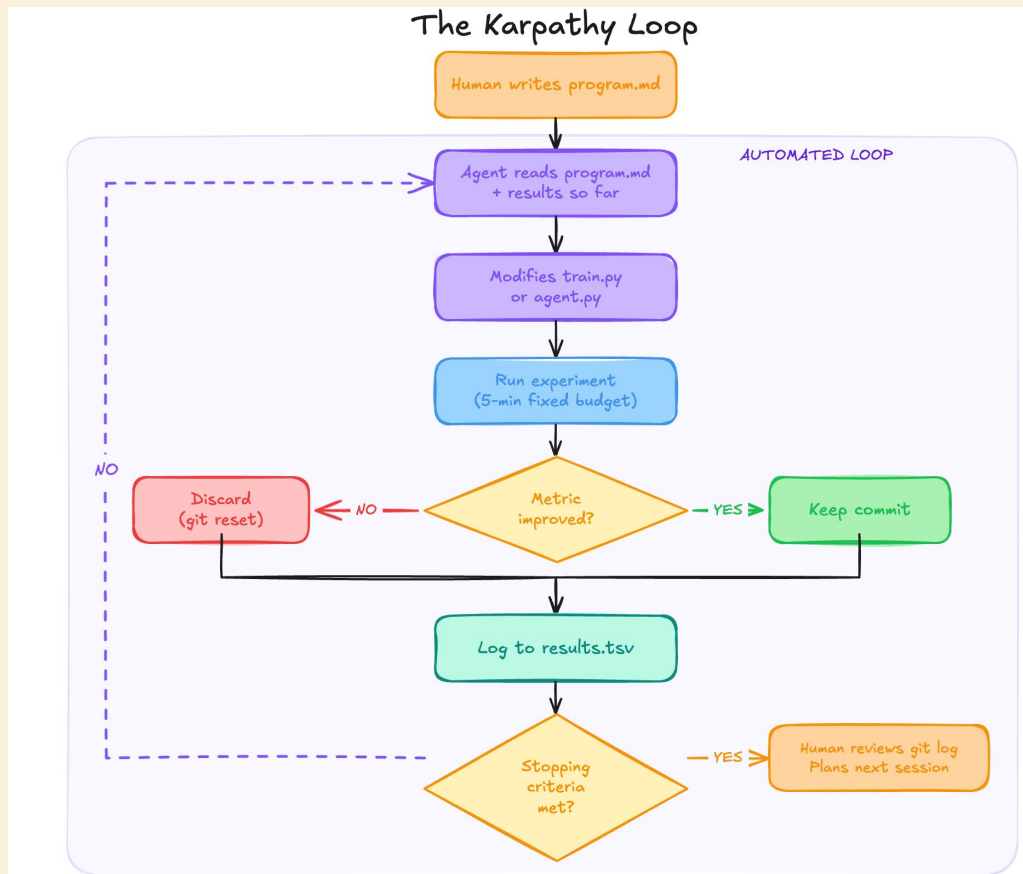
<https://openhands.dev/blog/20260219-velocity-is-dead>
<https://www.ronin.consulting/artificial-intelligence/using-the-ralph-wiggum-loop/>

Tool Loop for Continual Improvement



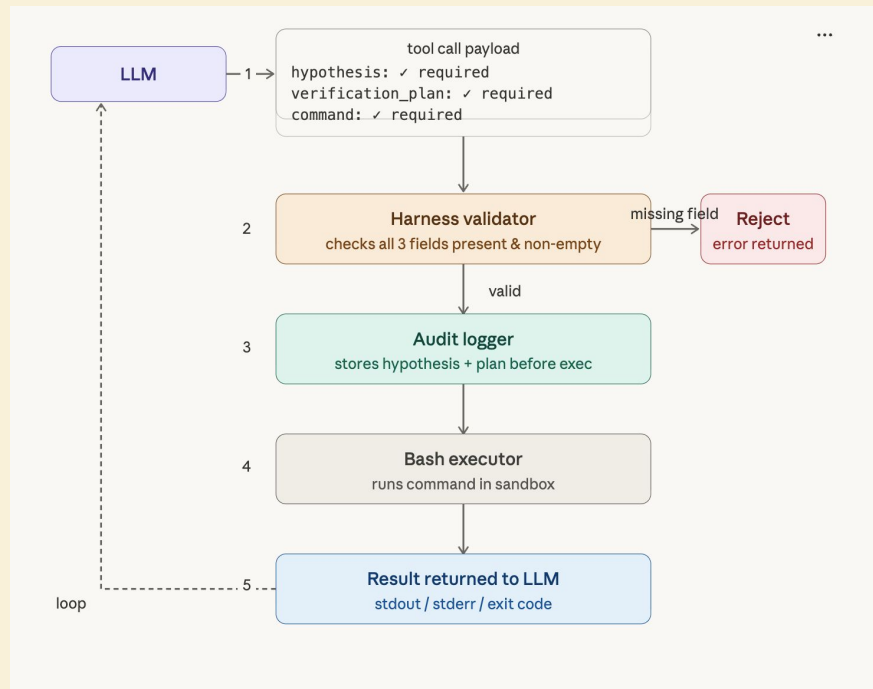
Tool Loop for Continual Improvement

<https://github.com/karpathy/autoresearch>

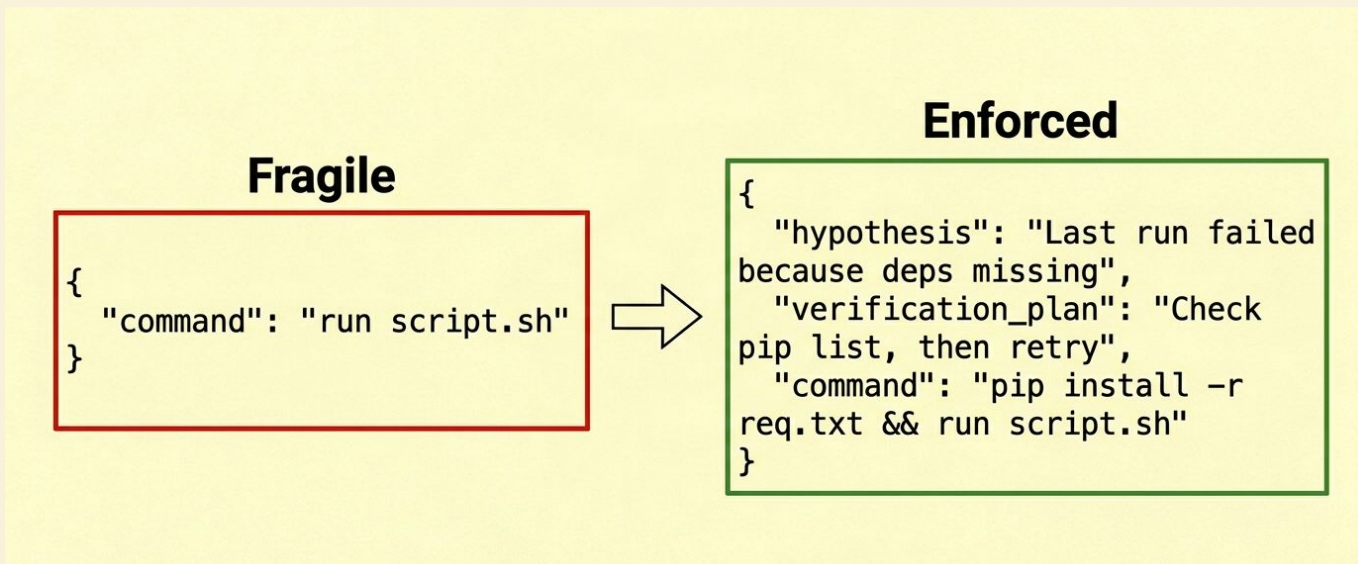


Cognitive Discipline via JSON Schema and Plan

Build a plan before you execute

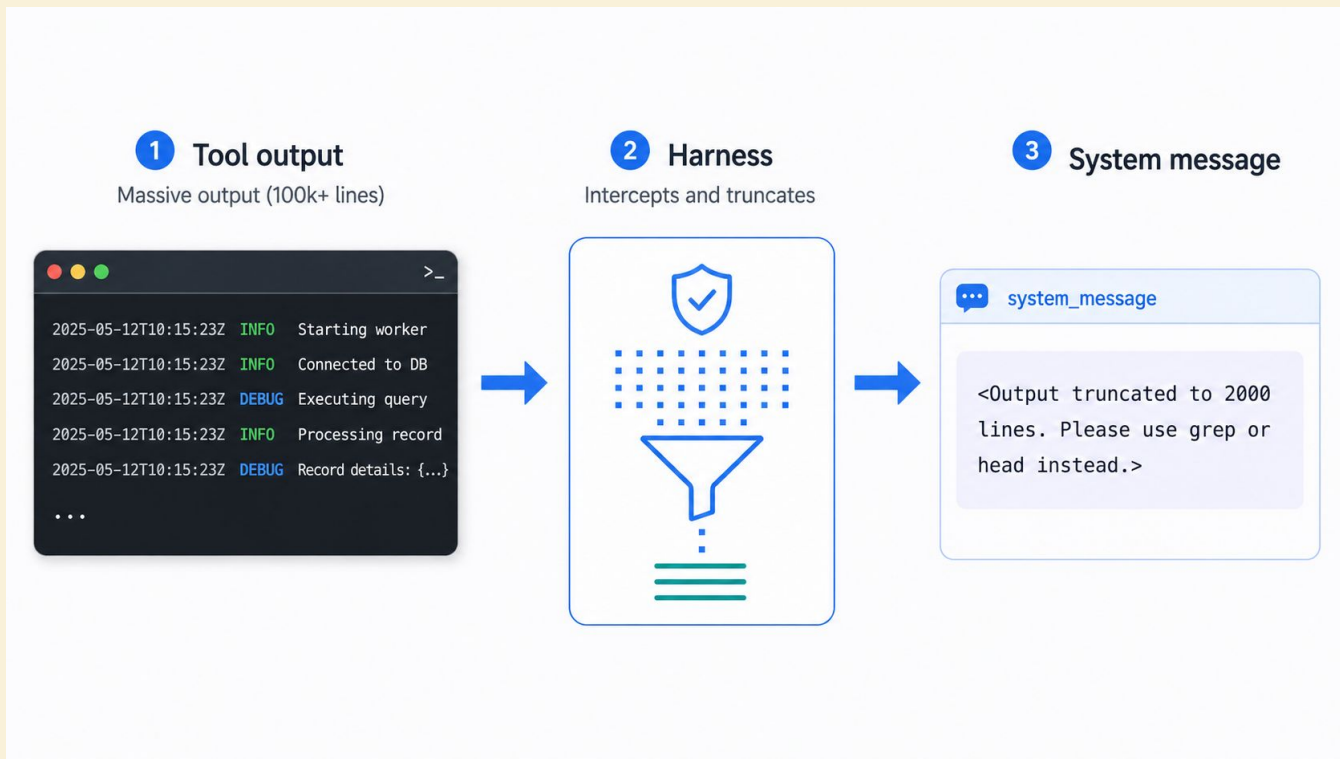


Externalize process



Protocols externalize the interaction burden.

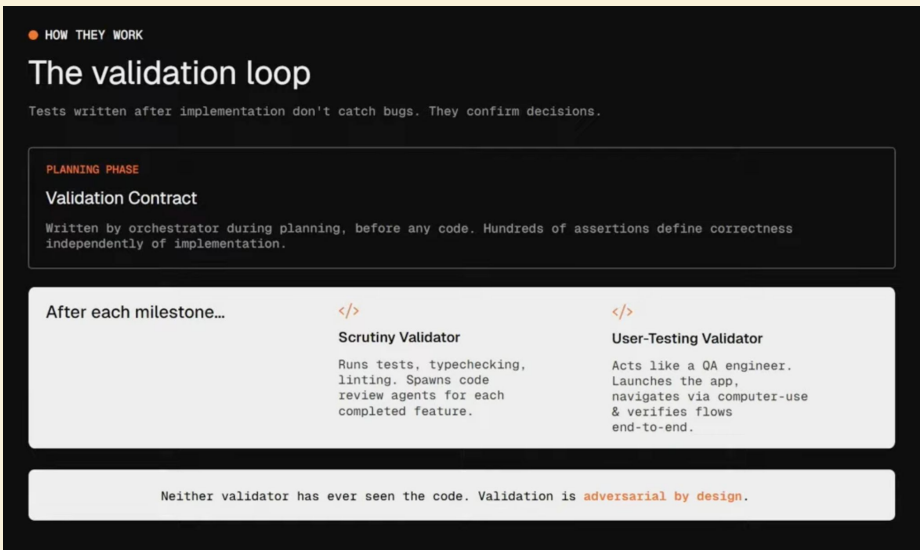
Defensive Tool Returns.



Environmental Discipline: Testing Driven Development

Tests written after implementation don't catch bugs; they just confirm the agent's decisions.

Enforce Validation Contracts, defining criteria before coding begins.



Adding System Constraints for your Harness

Code is Free. Architecture is Expensive.

Add system constraints:



Lint errors → instructions to the agent



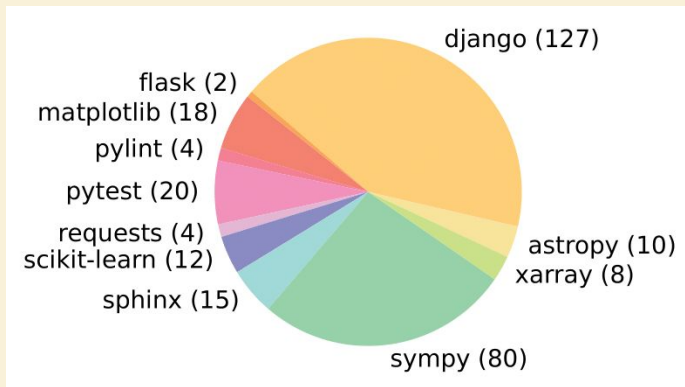
File-length tests → force decomposition



CI checks → the real guardrail

Principles for Agents

1. Actions should be simple and easy to understand for agents.
2. Actions should be compact and efficient.
3. Environment feedback should be informative but concise.
4. Guardrails mitigate error propagation and hasten recovery



Safety & Friction: Guardrails and Approval Friction

need to add the importance
need security



<https://cursor.com/blog/semsearch>

Guardrails and Approval Friction

Design friction to match blast radius.

Permission Ladder

Action class	Example	Policy
 Safe	read, grep, ls	Auto-allow
 Reversible	edit file, git commit	Auto-allow in sandbox
 Network	curl, npm install	Prompt once per session
 Destructive	rm -rf, DROP TABLE, force push	Require explicit approval

<https://www.anthropic.com/engineering/beyond-permission-prompts-making-claude-code-more-secure-and-autonomous>

- <https://www.anthropic.com/engineering/writing-effective-tools-for-agents>

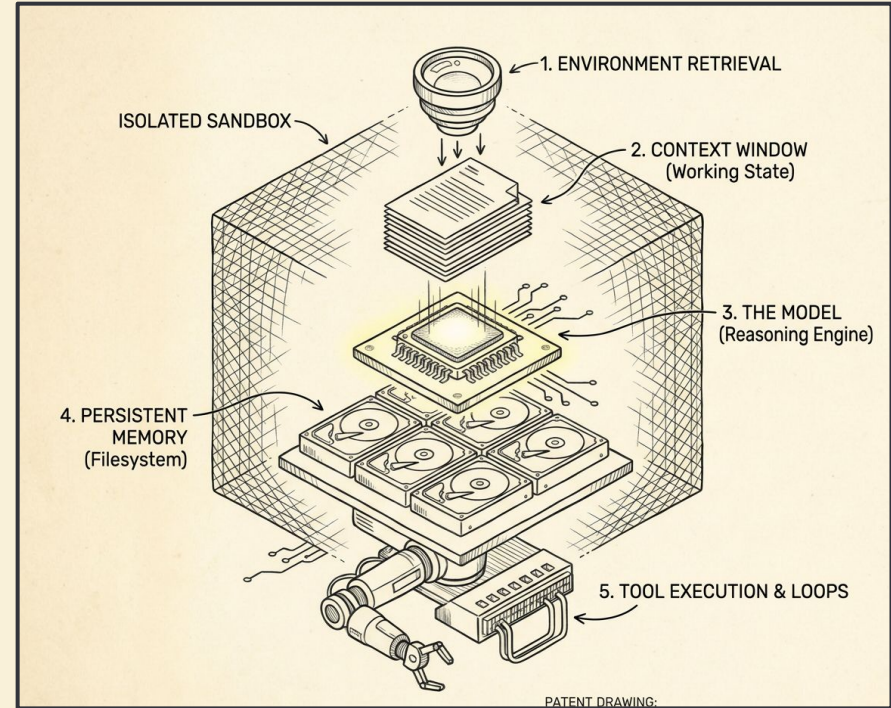
- <https://github.com/walkinglabs/awesome-harness-engineering>

Loop Design Rules

- Make the next action explicit
- Make verification cheap
- Make blind repetition hard
- Force learning from the environment.

Let's start with how agents find what they need.

- The Model
- Retrieval
- Context & Memory
- Agentic Loops & Tool Use
- System Architecture

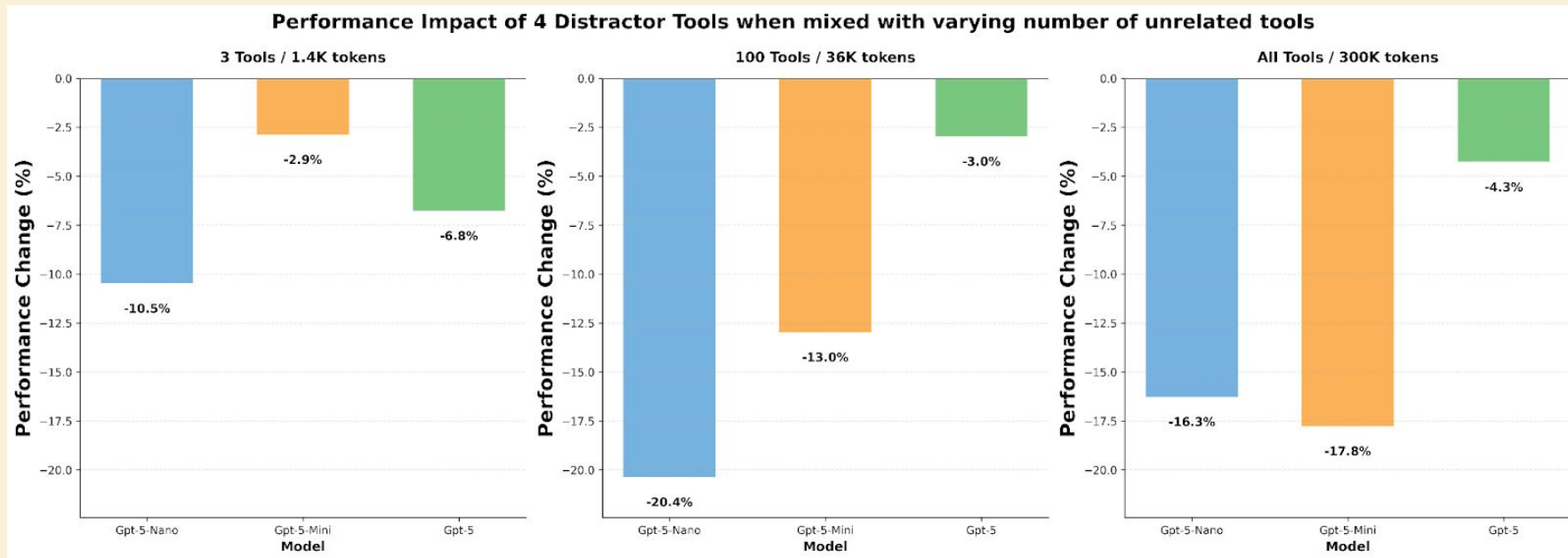


System Architecture: Single versus Multi Agent

Architecture shapes work, state, and failures.

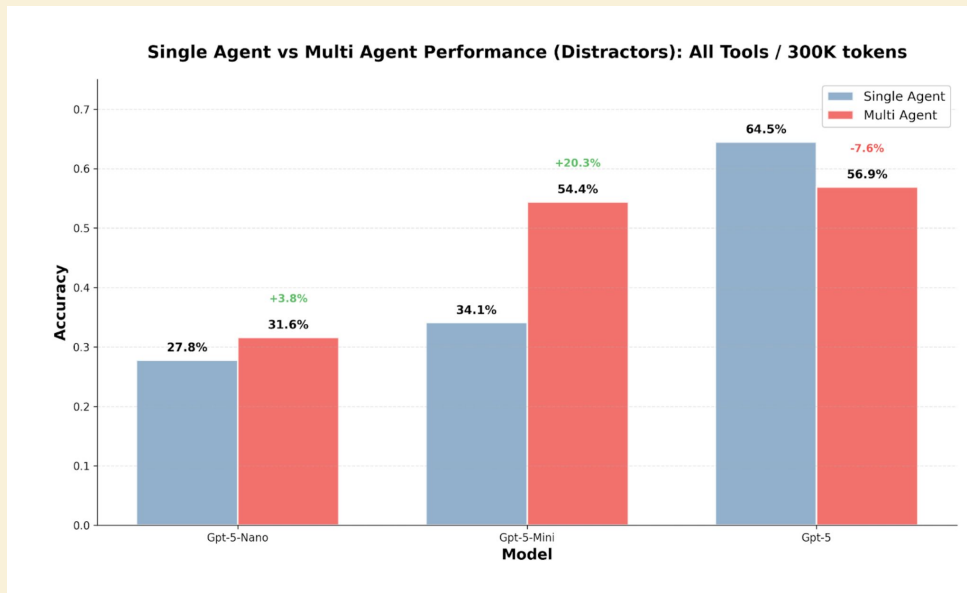
Who's tried a multi-agent in production?

Single agents degrade as complexity grows.



As context size and the number of tools increase, even the best models start to collapse. They don't know which tool to pick."

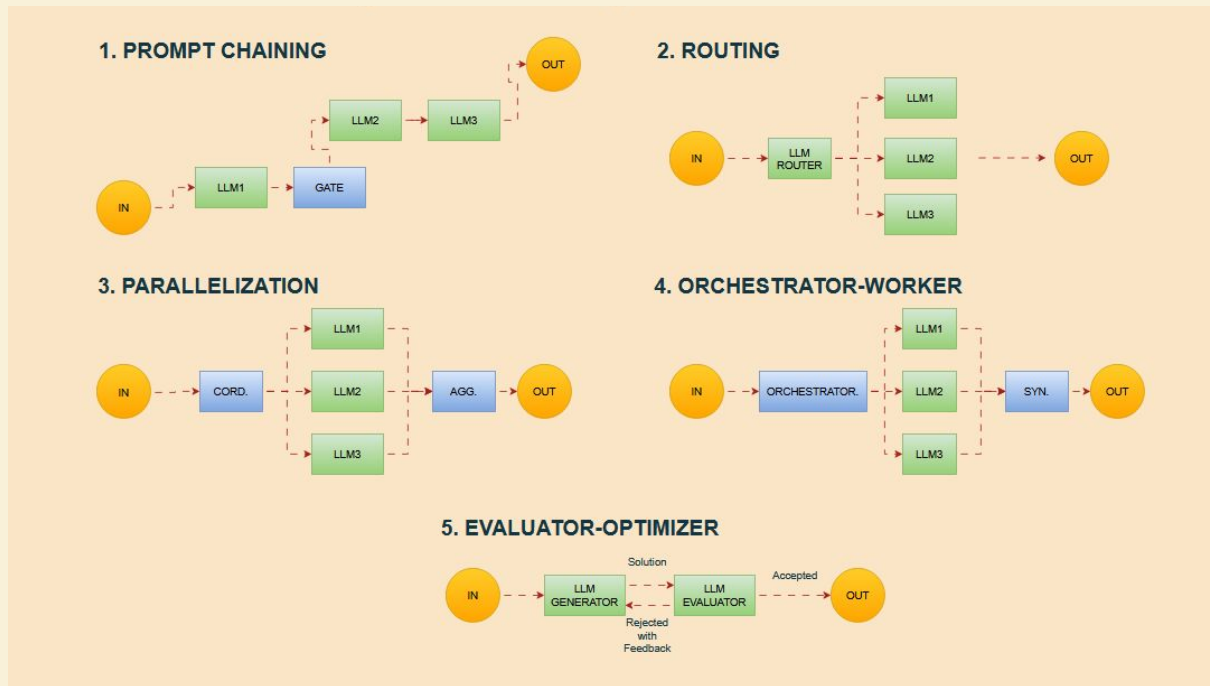
The Promise: Use subagents when one agent breaks.



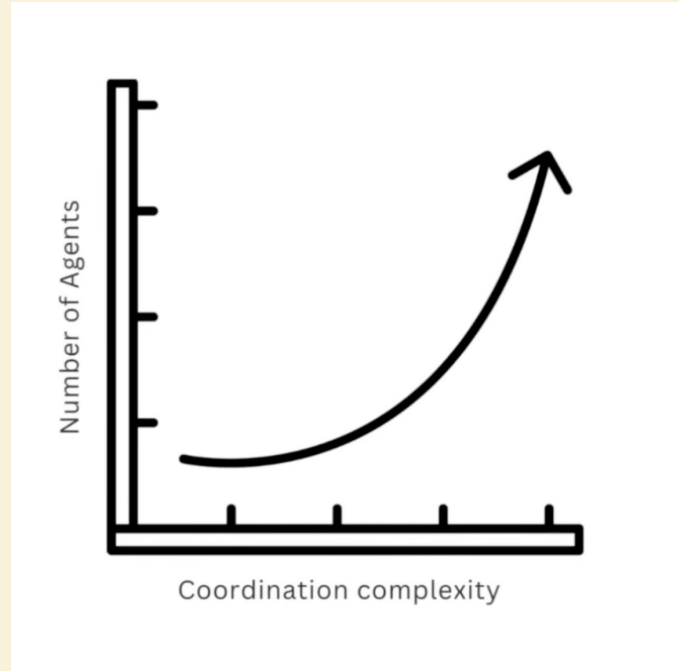
By splitting the context and tools, multi-agent systems can outperform single-agent baselines.

Subagents help when work splits cleanly.

You can use basic routing, parallelization, or an Orchestrator-Worker model

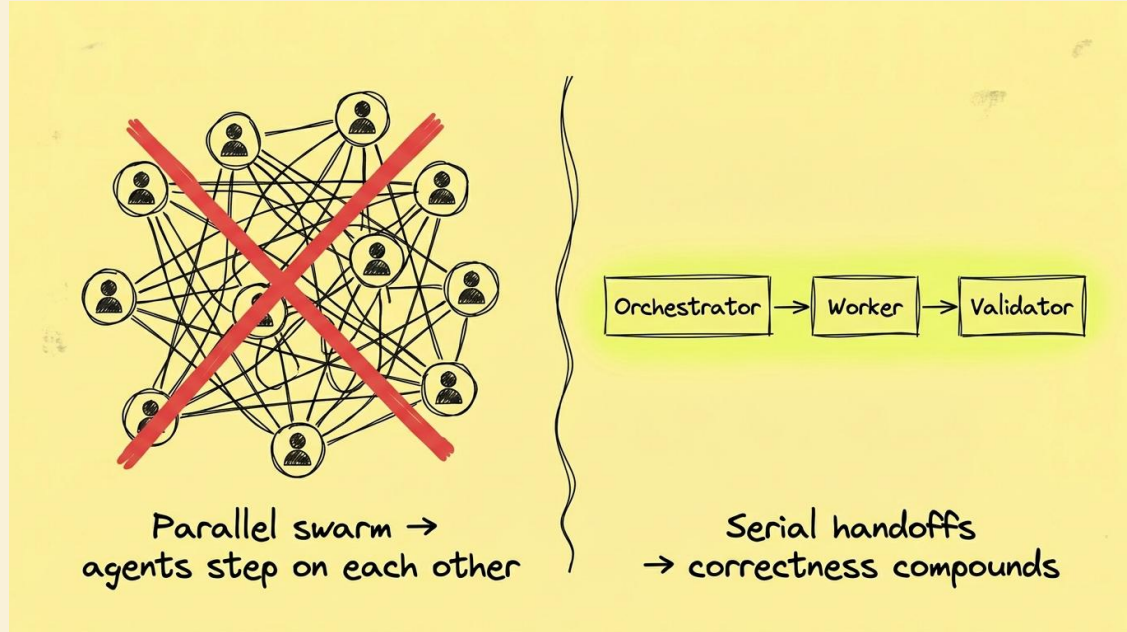


Multi-Agent is like Distributed Systems: Complex!



<https://www.youtube.com/watch?v=2czYyrTzILg>

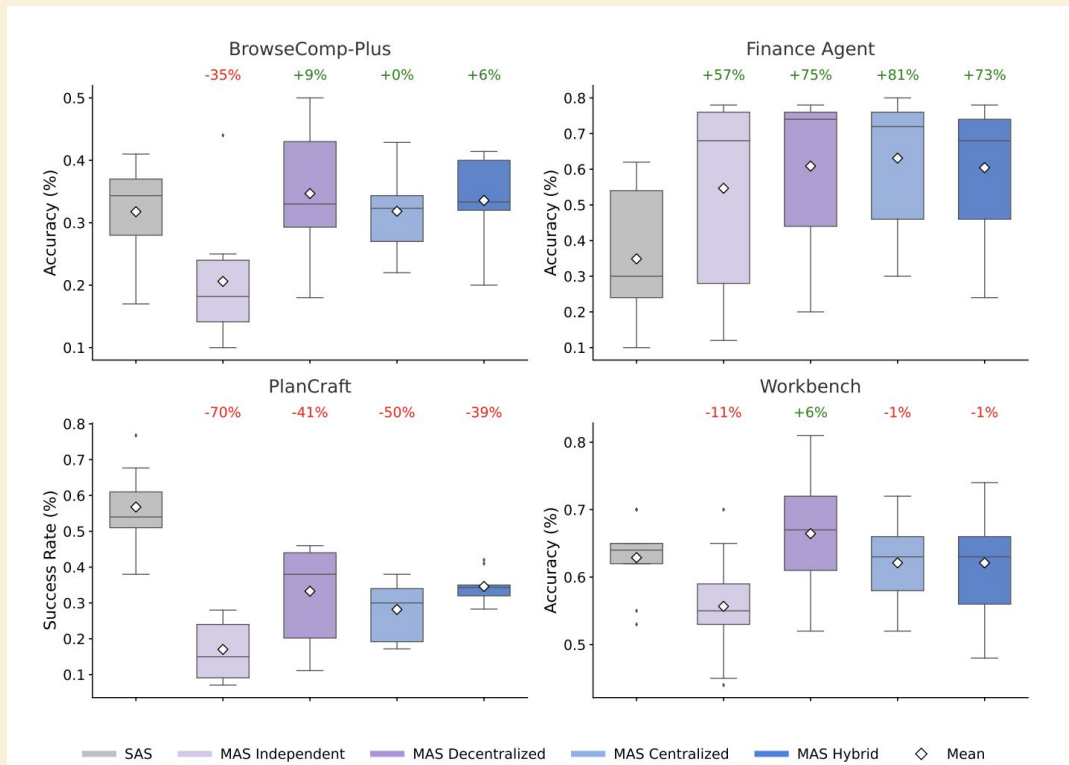
Coordination Tax - Going from Parallel to Serial



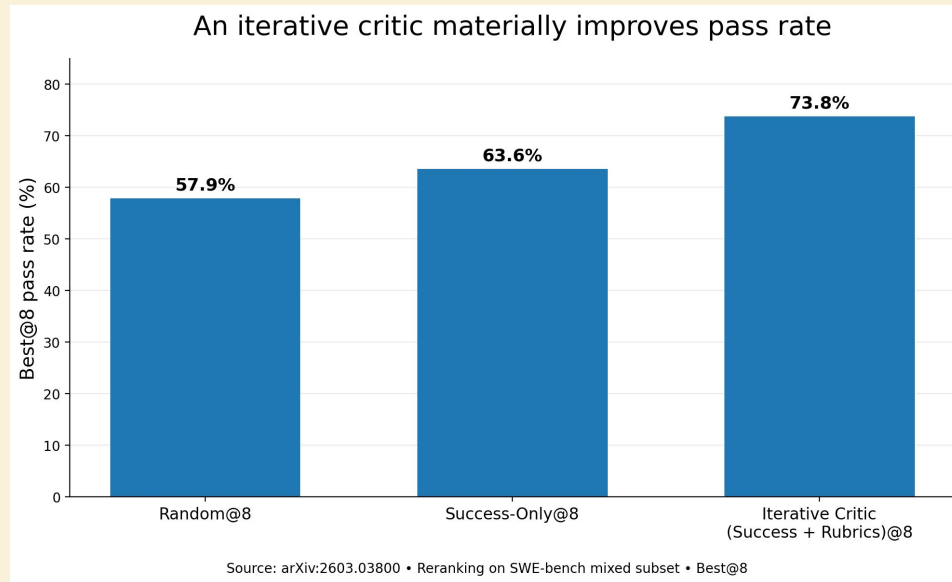
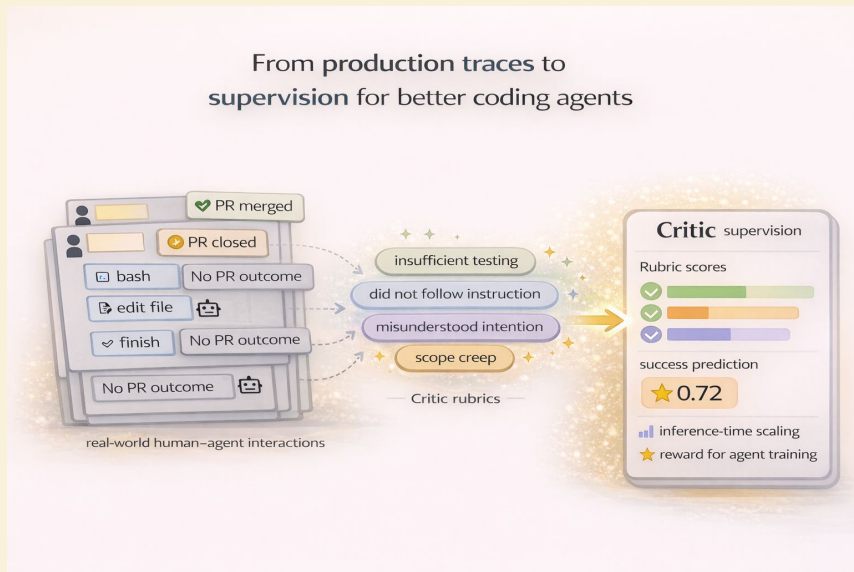
Every conflict burns tokens. Every retry erodes trust.

The Reality: More agents only help if coordination stays cheap.

- On average, multi-agent systems perform worse than single agents (-3.5%).
- Once a single agent reaches ~45% accuracy, adding agents stops helping. Coordination cost outweighs reasoning gains.
- Architecture determines if errors are corrected or amplified. Independent agents amplify errors ~17x.



Multi-Agent critics using reflection



Rubric-Supervised Critic <https://arxiv.org/pdf/2603.03800>

Reflexion: <https://arxiv.gg/abs/2303.11366>

Boris Cherny: 2 to 3X better quality

Is harness engineering just the new prompt engineering?

Commoditizing (will be default)	Durable (keeps mattering)
Compaction algorithms	Skills systems — your org's expertise
Default tool choices (grep, read, edit)	Memory policy — what persists, what doesn't
Model-specific prompt tweaks	Domain-specific tool libraries
Most "hacks" patching capability gaps	Security posture & approval friction
	Evals — knowing when your harness breaks

Closed or Open Harness?

1. Closed harnesses make decisions about models, memory, tools, security, and runtime,
2. Closed harnesses are not better than open ones
3. Open ones give you flexibility to control the performance of your coding agent